

Методы моделирования пользовательской активности на основе пуассоновских процессов

Алексей Мошкин

МКН СПбГУ

Научный руководитель: Николаев Максим Сергеевич

2026

Аннотация

Работа посвящена методам моделирования пользовательской активности в e-commerce на основе пуассоновских процессов. Прогноз пользовательской активности — важный блок при оценке ценности клиента, планировании коммуникаций с ним и подведении итогов A/B-экспериментов. Точные модели интенсивности позволяют не только улучшать бизнес-метрики, но и интерпретировать структуру пользовательского поведения. В работе ставится общая задача предсказания дневной интенсивности покупок пользователя $\lambda_{u,t}$ в предположении условной пуассоновской модели наблюдения и решается моделями разных классов — от простых пуассоновских моделей до многомерных процессов Хоукса и градиентного бустинга на расширенном наборе признаков. Основной фокус смещён в сторону применимости процессов Хоукса с экспоненциальным ядром: они естественно описывают самовозбуждение и кросс-канальные влияния событий и при этом сохраняют легко интерпретируемые параметры, в отличие от более сложных моделей машинного обучения.

В работе мы показываем, что Хоукс-паттерны в данных действительно присутствуют, однако из-за разреженности целевой переменной и эффекта «перетока массы» между бейзлайн-частью и Хоукс-надстройкой коэффициенты модели устойчиво проявляются только на достаточно длинных train-окнах. Кроме предсказательного качества Хоукс-моделей исследуются их сходимость, чувствительность к выбору регуляризации, идентифицируемость коэффициентов и поведение на разных длинах обучающего окна — аспекты, важные для практического применения модели.

В качестве данных используется датасет из 10К пользователей крупного маркетплейса за 290 дней с семью поведенческими счётчиками за день: число поисков, переходов из категории в корзину и к оформлению заказа, переходов из поиска в корзину и к оформлению, добавлений товара в корзину и оформленных заказов. Целевая пере-

менная — число оформленных заказов на пару пользователь–день. Все эксперименты проводятся на воспроизводимом коде на Python.

Основные результаты:

1. Построена иерархия из шести вероятностных моделей; на главном 207-дневном train-test разбиении Хоукс-надстройка устойчиво улучшает персонализированный пуассоновский бейзлайн на трёх независимых каналах.
2. Восстановлена cross-channel матрица Хоукс-взаимодействий $\alpha \in \mathbb{R}^{3 \times 3}$ для каналов воронки `searches` $\rightarrow$ `to_cart` $\rightarrow$ `to_ord` и оценены доверительные интервалы её коэффициентов. Через ландшафт лосса показано, что диагональные коэффициенты на разреженных каналах слабо идентифицируемы, что согласуется с известными результатами по многоядерным Хоукс-процессам [4, 5].
3. Так как данные табличные и наблюдений много, ожидаемо верхнюю планку качества дала бустинг-модель, обученная на расширенном синтетическом наборе признаков (построен 141 признак), — но с менее интерпретируемой структурой.
4. Показано, что эффект от Хоукс-ядер сильно скоррелирован с настройками базовой интенсивности: между Хоукс-частью и базовой интенсивностью возможен «переток массы» без значительной потери качества. Поэтому выбор регуляризации (численной или структурной) очень важен — именно он регулирует, где останется масса. Особенно сильно этот эффект проявляется на коротких train-test окнах, где в зависимости от регуляризации веса Хоукс-ядер могут полностью обнулиться.

Содержание

Аннотация	1
1 Введение	3
1.1 Постановка задачи и актуальность	3
1.2 Процессы Хоукса: специфика и применимость к задаче	3
1.3 Цели и задачи работы	5
1.4 Используемые данные	5
2 Обзор литературы	7
3 Разведочный анализ данных и метрика качества	9
3.1 Первичный анализ	9
3.2 Среднедневная интенсивность на всём периоде	10
3.3 Гетерогенность пользователей	10
3.4 Корреляции каналов и выбор признаков для Хоукс-надстройки	12
3.5 Метрики качества	13

4	Глава I. Бейзлайн и Хоукс-надстройка на полном train-test	14
4.1	Постановка: зачем нужен сильный бейзлайн	14
4.2	Построение бейзлайна	14
4.3	Scaled-baseline Hawkes	17
4.4	Joint Hawkes как альтернативная параметризация	20
4.5	Бустинг	21
4.6	Итоговая лестница моделей	22
4.7	Декомпозиция интенсивности: Scaled-baseline vs Joint Hawkes	24
5	Глава II. Поведение моделей на коротких train-окнах	26
5.1	Альтернативное train-test разбиение	26
5.2	Вырождение Scaled-baseline Hawkes	27
5.3	Joint Hawkes на коротких окнах	27
5.4	Анализ результатов в зависимости от длины train-test разбиения	28
6	Глава III. Структура cross-channel Хоукс-матрицы	31
6.1	Постановка и верификация cross-channel модели	31
6.2	Стабильность коэффициентов: train-окно и регуляризация	33
6.3	Влияние силы регуляризации на матрицу Joint Hawkes	34
6.4	Идентифицируемость коэффициентов: диагональ против внедиагональных	36
6.5	Доверительные интервалы коэффициентов и финальная матрица	39
7	Глава IV. Per-user визуализация работы моделей	42
8	Выводы	43
9	Заключение	45
10	Использование инструментов искусственного интеллекта	46

1 Введение

1.1 Постановка задачи и актуальность

Задача предсказания пользовательской активности — важная в современном e-commerce. Точная оценка ожидаемого числа покупок `to_ord` на пару `(user, day)` может напрямую использоваться в персонализации ранжирования, расчёте CLV и подведении итогов А/В-экспериментов.

С формальной точки зрения мы оцениваем дневную интенсивность $\lambda_{u,t}$ и предполагаем условную Пуассоновскую модель наблюдения:

$$y_{u,t} \mid \lambda_{u,t} \sim \text{Poisson}(\lambda_{u,t}),$$

где $y_{u,t}$ — число покупок пользователя u в день t , а $\lambda_{u,t} = \lambda_{u,t}(\text{data})$ — наша точечная оценка интенсивности, выдаваемая некоторой моделью по доступным к этому моменту данным. Задача состоит в предсказании $\lambda_{u,t}$ на отложенной тестовой выборке.

Особенность задачи — крайняя разреженность сигнала: в наших данных среднее число покупок на `(user, day)` ≈ 0.10 , а доля нулевых ячеек — 93.7%. Такие данные плохо подходят к классическим регрессионным методам, ориентированным на нормальные ошибки, и требуют явного учёта Пуассоновского распределения наблюдений.

Ещё одна особенность — событийный характер сигнала. Покупки не происходят равномерно во времени: часто наблюдаются «вспышки» активности (пользователь сделал серию заказов за один-два дня) с последующей длительной паузой. Для моделирования такой динамики могут применяться процессы с самовозбуждением, в которых недавние события явно влияют на интенсивность будущих событий.

Актуальность работы определяется двумя факторами. Во-первых, промышленные системы прогноза покупок чаще всего опираются на хорошо отлаженные GBDT и нейросети, и статистическая модель вряд ли сможет побить их по сырому качеству предсказания. Однако такая модель, как правило, стабильнее ведёт себя на коротких окнах и обладает абсолютно интерпретируемыми параметрами — что особенно важно при оценке «слабых эффектов» (например, в А/В-экспериментах). Во-вторых, процессы Хоукса, успешно применявшиеся в финансах [2] и анализе кликовых потоков [3, 7], редко рассматриваются в промышленном e-commerce как полноценный инструмент — несмотря на то, что их параметры (`reg-user` множитель, матрица α -взаимодействий) допускают прямую бизнес-интерпретацию.

1.2 Процессы Хоукса: специфика и применимость к задаче

Процесс Хоукса — это самовозбуждающийся точечный процесс, в котором интенсивность будущих событий зависит от истории уже случившихся [1]. В классической непрерывной форме одномерный процесс с экспоненциальным ядром записывается как

$$\lambda(t) = \mu + \sum_{t_i < t} \alpha \cdot \exp(-\beta(t - t_i)),$$

где μ — фоновая интенсивность, α — вес эффекта самовозбуждения, а β — скорость затухания сигнала. Каждое произошедшее к текущему моменту событие повышает интенсивность в ближайшем будущем. Память этого процесса можно считать виртуально конечной: после нескольких «периодов полураспада» событие фактически перестаёт влиять на текущую интенсивность.

В многомерной форме (K типов событий) ядра становятся матричными:

$$\lambda_k(t) = \mu_k + \sum_{j=1}^K \sum_{t_i^{(j)} < t} \alpha_{kj} \cdot \exp(-\beta_{kj}(t - t_i^{(j)})).$$

Здесь α_{kj} интерпретируется как «сила влияния события типа j на интенсивность типа k ». Эта матричная структура и есть та самая cross-channel матрица, которая в нашей работе восстанавливается для воронки `searches` → `to_cart` → `to_ord`.

В дискретно-дневной форме, удобной для нашей задачи, мы пишем

$$\lambda_{u,t} = \underbrace{\mu_{u,t}}_{\text{бейзлайн}} + \sum_j \alpha_j \cdot z_{u,j,t},$$

где $\mu_{u,t}$ — бейзлайн пользовательской интенсивности без учёта самовозбуждения (его модель описана далее), а $z_{u,j,t}$ — экспоненциально сглаженный счётчик j -го признака у пользователя u до дня t :

$$z_{u,j,t} = \sum_{s < t} x_{u,j,s} \cdot \exp\left(-\frac{t-s}{\tau_j}\right),$$

с заданным полураспадом τ_j (в большинстве экспериментов мы используем два ядра с полураспадами 1 и 3 дня одновременно). Веса α_j оцениваются по обучающей выборке и допускают прямую интерпретацию: положительный α_j означает, что событие признака j повышает интенсивность будущих покупок этого пользователя.

Применимость процессов Хоукса к нашей задаче объясняется следующими свойствами:

1. **Self-excitation естественно описывает воронку покупок.** В e-commerce покупка не происходит «из ниоткуда»: ей часто предшествует серия поисков, просмотров и добавлений в корзину. Hawkes явно моделирует эту короткую память.
2. **Параметры интерпретируемы.** Матрица $\alpha[\text{target} \leftarrow \text{source}]$ показывает, какие каналы и в какой мере возбуждают другие каналы. Это даёт бизнес-понимание, которое у GBDT выражено слабее.
3. **Аддитивная декомпозиция бейзлайн + надстройка.** Модель явно разделяет «фоновую активность» пользователя и «короткие всплески от событий», что

позволяет независимо изучать обе компоненты.

Платой за интерпретируемость становится более ограниченная функциональная форма: линейная аддитивная комбинация $\lambda_u \cdot b_t + \alpha^\top z$ игнорирует нелинейные взаимодействия признаков, которые более сложная модель могла бы учесть. Оценка того, насколько много полезной информации при таком упрощении мы теряем — одна из задач работы.

1.3 Цели и задачи работы

Цель работы: исследовать методы моделирования пользовательской активности на основе пуассоновских процессов — от простых пуассоновских бейзлайнов до многомерных процессов Хоукса — с акцентом на применимость Хоукс-моделей; оценить их предсказательное качество и стабильность по сравнению с бустингом, а также изучить идентифицируемость и интерпретируемость их коэффициентов.

Задачи работы:

1. Построить иерархию вероятностных пуассоновских моделей, дающую устойчивый бейзлайн для последующей Хоукс-надстройки.
2. Исследовать особенности сходимости коэффициентов Хоукс-моделей: на каком объёме обучающей выборки они стабилизируются и какая параметризация подходит для коротких окон.
3. Восстановить cross-channel матрицу $\alpha[\textit{target} \leftarrow \textit{source}]$ для трёх каналов воронки и оценить её устойчивость к выбору train-окна и регуляризации.
4. Сравнить полученные пуассоновские модели с бустингом на расширенном синтетическом наборе признаков — на нескольких train-test разбиениях.

1.4 Используемые данные

Все эксперименты проводятся на открытом датасете, обезличенно отражающем поведение 10000 случайно выбранных пользователей крупного e-commerce-сервиса за период 2025-01-15..2025-10-31 (290 дней). На каждую пару (`user`, `day`) доступны семь поведенческих счётчиков (число событий за день):

- `searches` — пользовательские поиски, среднее ≈ 1.20 событий на строку;
- `cat_to_cart` — переходы из категории в корзину;
- `cat_to_ord` — переходы из категории к оформлению заказа;
- `to_cart` — добавления товаров в корзину, среднее ≈ 0.37 ;
- `to_ord` — оформленные заказы (целевая переменная), среднее ≈ 0.10 ;

- также есть счётчики `search_to_cart`, `search_to_ord`. Но они на 98% скоррелированы с `to_cart`, `to_ord`. Дополнительного качества они не добавляют, зато усложняют идентификацию весов. Активно пользоваться ими при обучении мы не будем.

Структура данных — длинный pandas DataFrame с ~ 2.87 М строк формы `(user_id, event_date, searches, to_cart, to_ord, ...)`.

2 Обзор литературы

Процессы Хоукса как класс самовозбуждающихся точечных процессов введены Хоуксом [1] и долгое время применялись преимущественно в сейсмологии и финансах [2]. С развитием современных систем сбора пользовательских данных интерес к ним смещается в сторону анализа кликовых потоков, рекомендательных систем и e-commerce.

Существенная часть современной литературы посвящена нейронным расширениям Хоукс-моделей. Du et al. [3] предложили recurrent marked temporal point processes (RMTPP), в которых дискретная история событий пользователя сжимается рекуррентной сетью в вектор состояния, по которому затем предсказывается интенсивность будущих событий. Mei и Eisner [7] развили эту идею в neural Hawkes process, явно параметризовав интенсивность нейросетью, сохраняющей формальную структуру процесса с самовозбуждением. Эти подходы дают высокое качество, но ценой снижения интерпретируемости: нейросетевой блок становится чёрным ящиком, и непосредственно «прочитать» матрицу cross-channel влияний из обученной модели нельзя. К тому же в нашей задаче нейросетевые методы плохо применимы по практическим причинам: данных по отдельным пользователям мало, риск переобучения высок, а регуляризацию у больших архитектур сложнее контролировать, чем у компактной параметрической модели Хоукса.

Параллельно развиваются классические многомерные Хоукс-модели с акцентом именно на интерпретируемость и идентифицируемость. Vasry, Vompaire, Gaïffas и Muzy [4] показывают, что при оценке коэффициентов многомерного Хоукс-процесса с помощью максимального правдоподобия имеет смысл регуляризовать диагональные элементы матрицы влияния отдельно от внедиагональных: первые отвечают за «своё» самовозбуждение канала и часто коллинеарны фоновой интенсивности μ , вторые описывают cross-channel взаимодействия и идентифицируются устойчивее. В нашей работе мы наблюдаем эту же асимметрию эмпирически (см. главу III) и связываем её с тем же механизмом коллинеарности history-state и бейзлайна.

В индустриальном контексте предсказания пользовательской активности классическим инструментом остаются модели семейства Pareto/NBD и Beta-Geometric, восходящие к работам Fader и Hardie [6]. Их центральная идея — Gamma-prior на индивидуальный темп покупок пользователя λ_u , с регуляризацией оценки через Empirical-Bayes posterior. Этот же приём используется нами в Personalized Gamma-Poisson бейзлайне и оказывается основным шагом, дающим крупнейший выигрыш по отрицательному логарифму правдоподобия (negative log-likelihood, NLL): модель аккуратно ранжирует пользователей по их интенсивности покупок и обеспечивает устойчивую базу для последующей Хоукс-надстройки.

Наконец, для сравнения с интерпретируемой моделью используются современные бустинговые методы, в первую очередь Gradient Boosting Machines в реализации Friedman [8] и их продвинутые версии (HistGradientBoosting, LightGBM, CatBoost). На широком наборе признаков такие модели дают верхнюю планку по качеству, особенно

когда сигнал содержит нелинейные взаимодействия признаков, плохо схватываемые линейной аддитивной формой Hawkes. Однако они слабее интерпретируемы: не дают компактных структурных коэффициентов, сопоставимых с матрицей α Хоукс-процесса, и поэтому используются в работе как контрольная модель.

3 Разведочный анализ данных и метрика качества

3.1 Первичный анализ

Панель данных представляет собой плотную таблицу ($\text{user} \times \text{day}$) без пропусков: для каждого из 10К пользователей доступны строки на все 290 дней анализируемого периода (нулевые счётчики записаны явно).

Распределение целевой переменной `to_ord` крайне разреженное. Гистограмма числа покупок на `user-day` ($\log\text{-Y}$) показана на рис. 1.

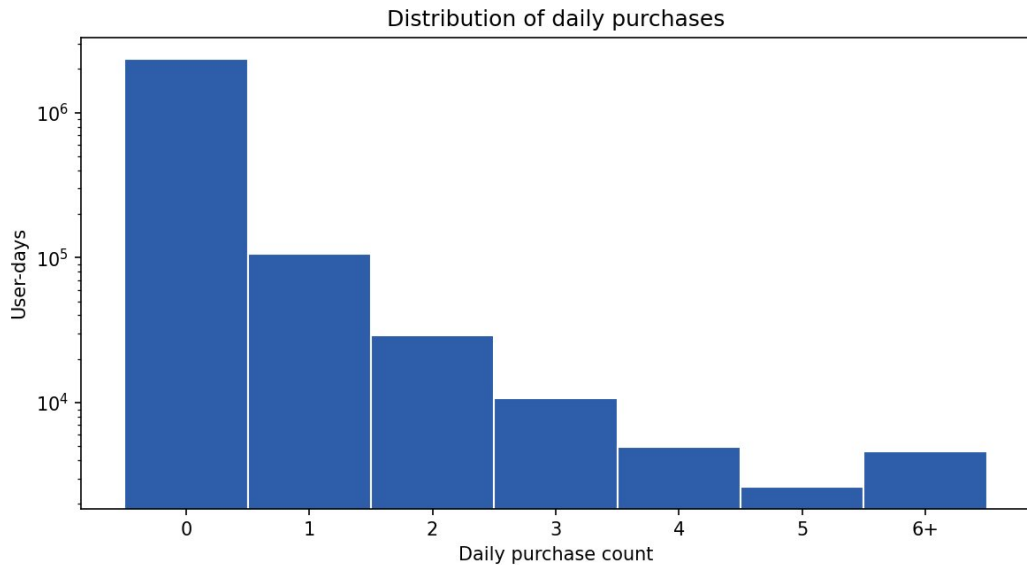


Рис. 1: Распределение числа покупок на `user-day` ($\log\text{-Y}$)

Видны три структурных факта:

- Подавляющее большинство ячеек содержат нули — точнее, 93.7% строк датасета имеют `to_ord = 0`.
- Существует тяжёлый правый хвост: единичные (`user`, `day`)-пары содержат десяток покупок и больше, при максимуме порядка сотни. Это означает, что распределение `to_ord` имеет крайне тяжелые хвосты.
- При этом среднее по данным ≈ 0.10 — и есть та самая разреженность сигнала.

Эта картина мотивирует две вещи. Во-первых, в нашей задаче число покупок пользователя естественно моделировать пуассоновским распределением — то есть наша задача сводится к предсказанию интенсивности покупок. Во-вторых, как-то учитывать различие между пользователями через персонализацию.

Распределение активности по пользователям тоже крайне неравномерное: 18% самых активных пользователей делают 62% всех покупок.

3.2 Среднедневная интенсивность на всём периоде

Агрегированная по дням дневная интенсивность `to_ord` представлена на рис. 2.

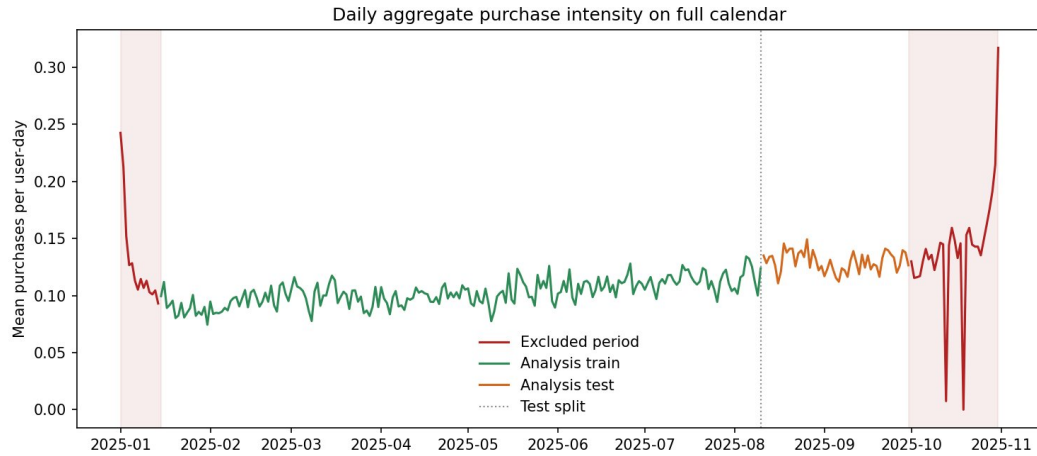


Рис. 2: Дневная интенсивность `to_ord` на полном ряду

На полном ряду выделяются три режима:

1. **Первые две недели января** — аномально высокая активность, связанная с послепраздничными промо-кампаниями. Среднее число покупок в этот период на 30..40% выше, чем в основном окне.
2. **Период с середины января до конца сентября** — основное стабильное окно с медленным трендом и недельной сезонностью. На нём проводится большая часть экспериментов.
3. **Октябрь** — повторное повышение уровня, связанное с предновогодними промо.

Так как данных за прошлые годы у нас нет, анализировать окна с сильно скачущей интенсивностью сложно. Основное окно для экспериментов фиксировано как 2025-01-15..2025-09-30, чтобы исключить новогодний всплеск из `train`-данных и обеспечить стационарность фоновых характеристик. Внутри этого окна также наблюдается недельная сезонность: будни и выходные различаются по среднему уровню `to_ord` примерно на 10%. Эту сезонность нужно будет учесть при построении бейзлайна.

3.3 Гетерогенность пользователей

Если в среднем по датасету число покупок ≈ 0.10 , то на уровне отдельного пользователя картина может быть сильно иной. Чтобы понять, насколько активность одного пользователя предсказуема через активность того же пользователя в другом периоде, мы построили облако суммарного числа покупок на `train` против `test` для каждого пользователя (рис. 3).

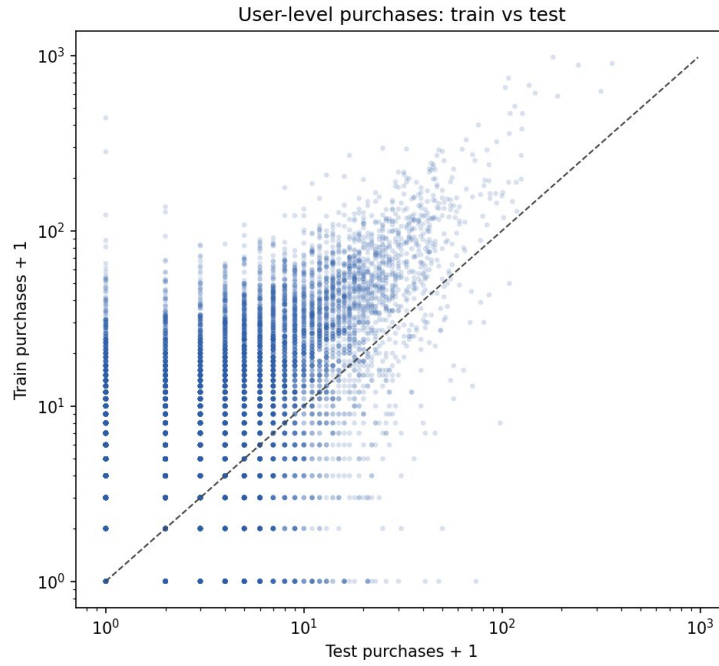


Рис. 3: Активность пользователей: train vs test

Корреляция Пирсона между train и test суммами в log-шкале (в которой и построен график) составляет 0.55; Spearman даёт близкое значение. То есть историческая активность предсказывает будущую заметно, но далеко не идеально — значительная часть дисперсии остаётся за счёт изменения паттерна пользователя во времени, шума и редких всплесков. И возможно изменение паттерна поведения как раз получится объяснить Хоукс-процессами.

Для иллюстрации структуры данных на уровне одного пользователя — пользователь 612605, активный (321 покупка на train) — показаны его дневные счётчики по трём каналам воронки на рис. 4.

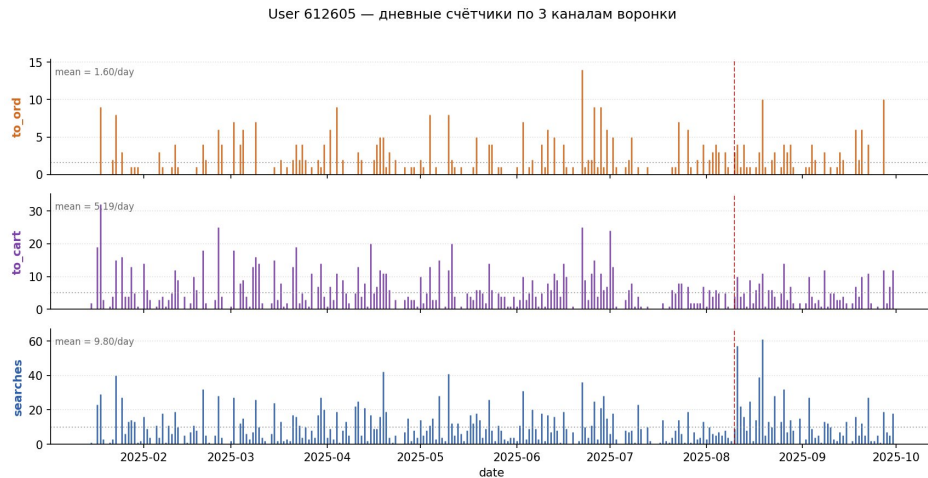


Рис. 4: Счётчики пользователя 612605 по 3 каналам

Активность нерегулярна — серии активных дней чередуются с периодами относительного затишья. Именно эту структуру самовозбуждения призваны моделировать

Хоукс ядра.

3.4 Корреляции каналов и выбор признаков для Хоукс-надстройки

Матрица корреляций семи исходных счётчиков показана на рис. 5.

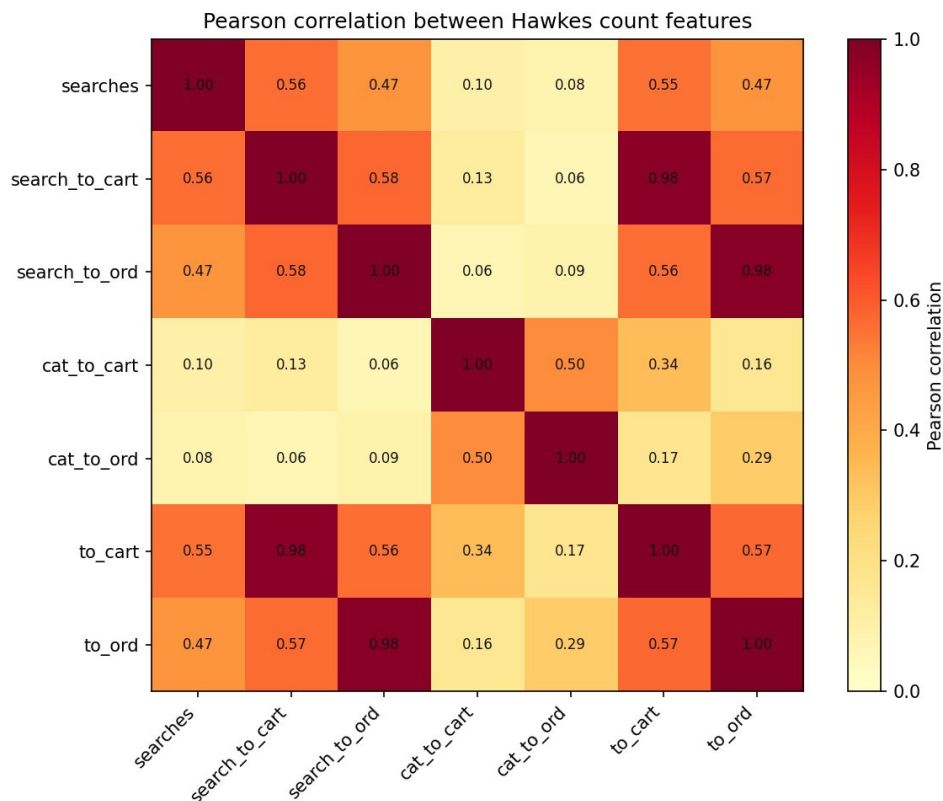


Рис. 5: Корреляция исходных каналов

Каналы `search_to_cart` и `search_to_ord` сильно скоррелированы с целевыми `to_cart` и `to_ord` (Пирсон $r > 0.9$), поэтому их включение в Хоукс-ядра привело бы к коллинеарности признаков и нестабильности оценок α . Также в датасете есть каналы `cat_to_cart` и `cat_to_ord`; информации в них мало, их добавление в Хоукс-модели не влияет на качество, зато усложняет анализ, поэтому их мы тоже не включаем. По итогам этого анализа в основной части работы мы оставляем три независимых канала: `searches`, `to_cart`, `to_ord`. Этого достаточно для содержательного cross-channel анализа и не приводит к мультиколлинеарности.

Корреляции между оставшимися тремя каналами умеренные ($r \approx 0.3..0.5$), что и должно быть для каналов воронки: события вышестоящих этапов (`searches`) частично, но не полностью предсказывают события нижестоящих (`to_ord`).

3.5 Метрики качества

Основная скоринговая метрика — средний минус log-likelihood Пуассоновской модели на одну строку датасета:

$$\text{NLL} = \frac{1}{N} \sum_{u,t} \left[\lambda_{u,t} - y_{u,t} \log \lambda_{u,t} + \log(y_{u,t}!) \right].$$

Чем меньше — тем лучше. Метрика согласована с самой моделью наблюдения и поэтому используется как основной критерий при сравнении моделей. Дополнительно для наглядной диагностики качества в исходной шкале счётчиков приводятся MAE и RMSE.

Очевидно, что нулевой NLL никакой Пуассоновской моделью получить нельзя — даже идеальное предсказание $\lambda = y$ в каждой строке даёт ненулевое значение из-за самой формы Пуассоновского правдоподобия. Этот теоретический минимум — saturated Poisson floor и для нашего test-окна ($N \approx 513K$ строк) он равен $NLL_{\text{sat}} \approx 0.0862$. Сам по себе этот floor недостижим: разрыв между ним и нашей лучшей моделью имеет две природы — часть его объясняется ограниченностью самой модели, не извлекающей весь доступный сигнал, а часть — это алеаторная неопределённость, шум в данных, который никаким Пуассоновским предсказанием закрыть нельзя.

4 Глава I. Бейзлайн и Хоукс-надстройка на полном train-test

В этой части мы поэтапно строим модель интенсивности $\lambda_{u,t}$. Сначала собираем сильный пуассоновский бейзлайн и обосновываем, какой именно его вариант лучше всего подходит как основа для Хоукс-надстройки. Затем строим саму Хоукс-надстройку и измеряем её вклад. Отдельно рассматриваем бустинг как контрольную модель, дающую верхнюю планку качества. В конце сводим все полученные модели в единую лестницу и сравниваем по нескольким метрикам. Все эксперименты этой части — на основном протоколе: train 207d, test 52d.

4.1 Постановка: зачем нужен сильный бейзлайн

Напомним постановку из раздела 1.2: в дискретной форме интенсивность раскладывается на бейзлайн и Хоукс-надстройку,

$$\lambda_{u,t} = \underbrace{\mu_{u,t}}_{\text{бейзлайн}} + \underbrace{\sum_j \alpha_j \cdot z_{u,j,t}}_{\text{Хоукс-надстройка}} .$$

Чтобы честно измерить вклад Хоукс-части, нужен максимально сильный бейзлайн $\mu_{u,t}$. Иначе любое улучшение от Хоукс-надстройки рискует объясняться тем, что бейзлайн просто плохо ловит ту часть сигнала, которую сами события могли бы объяснить и без явной структуры самовозбуждения. Поэтому в Главе I сначала строим бейзлайн поэтапно — от простейшего «единая константа» до Personalized Gamma-Poisson с байесовской регуляризацией per-user множителя, — и только после этого добавляем над ним Хоукс-надстройку.

4.2 Построение бейзлайна

Бейзлайн строится поэтапно — сначала несколько простых уровней без персонализации, потом полноценная пользовательская модель:

1. **Global Poisson** — единая константа $\bar{\lambda}$ на весь датасет. Самая простая модель; даёт точку отсчёта.
2. **Rolling Poisson** — скользящее среднее дневного уровня по предыдущим дням. Добавляет учёт глобального тренда без персонализации.
3. **Rolling Seasonal Poisson** — Rolling с поправкой на день недели. Учитывает недельную сезонность (будни vs выходные).
4. **Personalized Gamma-Poisson** — переход от общего дневного профиля к модели с индивидуальным множителем μ_u для каждого пользователя. Эта модель станет финальным бейзлайном для Хоукс-надстройки.

На нашем протоколе первые три модели различаются между собой слабо (test NLL $0.4608 \rightarrow 0.4576 \rightarrow 0.4576$): разброс активности между пользователями кратно превышает изменчивость дневного среднего и недельной сезонности. Большой шаг — это переход к персонализации, дающий test NLL 0.4096 (выигрыш -0.051 в NLL над Global Poisson). Ниже подробно разбираем именно Personalized GP как опорный бейзлайн для всех Хоукс-моделей.

Идея персонализации. Дневной профиль b_t из Rolling Seasonal даёт хорошее общее представление об уровне активности по когорте, но не отличает активного пользователя от неактивного. Естественный способ это исправить — домножить общий профиль на индивидуальный коэффициент μ_u :

$$\lambda_{u,t} = \mu_u \cdot b_t.$$

Если бы у каждого пользователя было много данных, μ_u можно было бы оценить простым MLE: $\hat{\mu}_u = Y_u/E_u$, где $Y_u = \sum_t y_{u,t}$ — суммарное число покупок пользователя на train, а $E_u = \sum_t b_t$ — суммарная экспозиция. Но в нашей задаче для большинства пользователей Y_u маленькое (0-2 покупки на train), и MLE-оценка получается крайне неустойчивой: один лишний наблюждённый день меняет $\hat{\mu}_u$ в разы. В крайнем случае при $Y_u = 0$ MLE даёт $\hat{\mu}_u = 0$, что приводит к нулевому прогнозу интенсивности и взрыву NLL на любом ненулевом дне теста.

Поэтому персонализацию в чистом виде применять нельзя — нужна регуляризация: малоактивные пользователи, про которых известно мало, должны автоматически подтягиваться к когортному уровню, а активные с достаточным количеством данных — оставаться близко к своей MLE-оценке.

Полный байесовский вывод. Модель достаточно простая, чтобы вместо явного штрафа провести регуляризацию через полный байесовский вывод. Зададим на μ_u Gamma-prior со средним ≈ 1 — то есть формализуем априорное представление о том, что средний пользователь имеет множитель около единицы:

$$y_{u,t} \mid \mu_u \sim \text{Poisson}(\mu_u \cdot b_t), \quad \mu_u \sim \text{Gamma}(\alpha, \beta).$$

Благодаря сопряжённости Gamma и Poisson posterior μ_u тоже распределён по Gamma и выписывается в замкнутой форме:

$$\mu_u \mid Y_u, E_u \sim \text{Gamma}(\alpha + Y_u, \beta + E_u).$$

В качестве точечной оценки берём среднее posterior-распределения:

$$\hat{\mu}_u^{\text{EB}} = \frac{\alpha + Y_u}{\beta + E_u}.$$

В этом выражении видно, как байесовский вывод сам решает задачу регуляризации. Для пользователя, про которого мало данных ($Y_u \approx 0$, E_u маленькое), оценка сводится к $\alpha/\beta \approx 1$ — то есть к среднему по когорте. Для активного пользователя с

большим Y_u влияние prior 'а становится пренебрежимо малым, и $\hat{\mu}_u^{\text{EB}} \rightarrow Y_u/E_u$, то есть к обычному MLE. Никакого дополнительного гиперпараметра «силы регуляризации» здесь нет — баланс между prior 'ом и данными выставляется автоматически в зависимости от того, сколько информации есть про конкретного пользователя.

Оценка гиперпараметров. Параметры (α, β) prior 'а оцениваются эмпирически — по marginal MLE по всем $\sim 10\text{K}$ пользователям одновременно (Empirical Bayes). Интегрируя μ_u , получаем маргинальное распределение (Y_u, E_u) — отрицательное биномиальное, и минимизируем суммарный $-\log p(Y_u, E_u \mid \alpha, \beta)$ по (α, β) методом L-BFGS-B. На нашем датасете это даёт $(\alpha, \beta) \approx (0.88, 0.88)$ — то есть prior со средним ≈ 1 , что согласуется с интерпретацией «средний пользователь имеет множитель около единицы».

Без байесовской регуляризации (чистая MLE с обрезкой $\mu_u = 0$ при $Y_u = 0$) test NLL на полном 52d-test составляет 0.4650. Полный байесовский вывод даёт 0.4096 — выигрыш порядка -0.055 в NLL. Это и есть основное улучшение на пути к опорному бейзлайну.

Анализ выигрыша на уровне отдельных пользователей через **per-user delta-LL** даёт следующую картину (рис. 6):

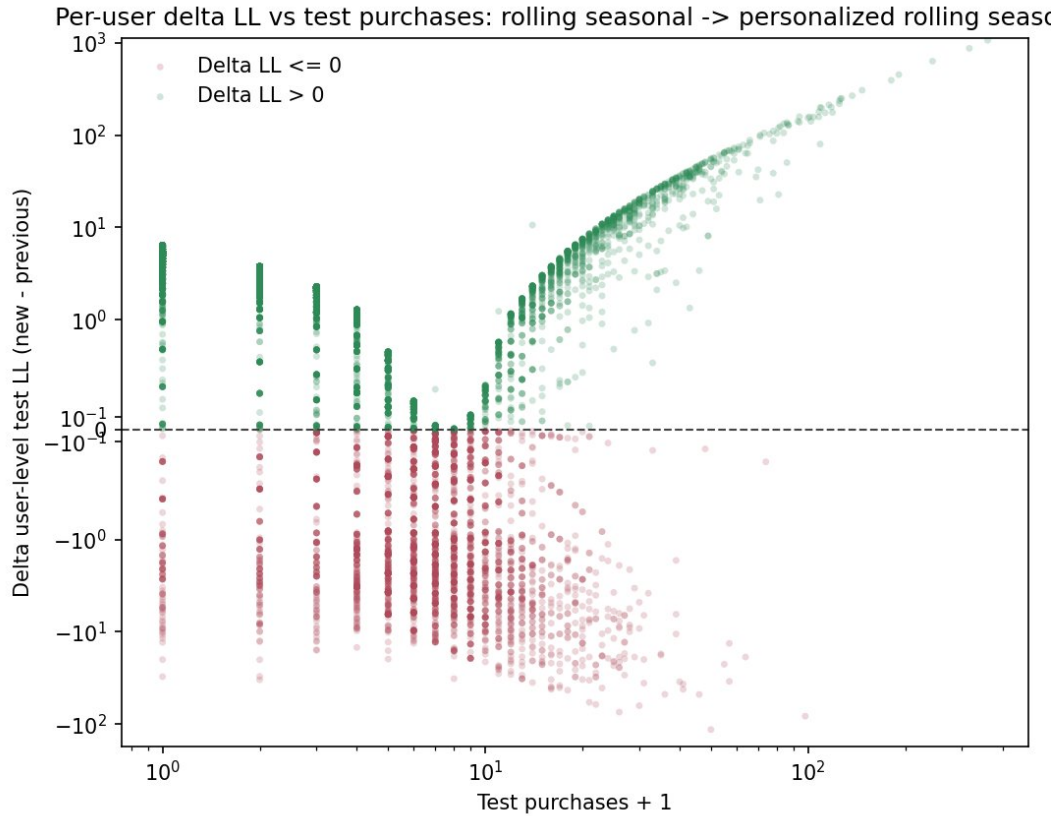


Рис. 6: Per-user delta-LL: Rolling Seasonal $\rightarrow$ Personalized

Распределение $\text{Delta-LL} = \text{LL}_{\{\text{PersGP}\}} - \text{LL}_{\{\text{RS}\}}$ по пользователям сильно неоднородно:

- $\text{share}(\text{personalized} > \text{rolling seasonal}) = 66\%$ — две трети пользователей выигрывают от персонализации.

- Юзеры с 0..2 покупками выигрывают почти всегда (87..94%) — для них Personalized GP правильно сдвигает прогноз вниз от среднего по когорте.
- Бакет 6..10 покупок — наоборот, проигрывает (18%): на этих пользователях регуляризация ещё ощутимо тянет к когортному среднему, и чистая MLE сработала бы лучше.
- Активные с 11+ снова выигрывают, потому что $\hat{\mu}_u^{\text{EB}}$ на них почти не отличается от MLE.

4.3 Scaled-baseline Hawkes

Над Personalized Gamma-Poisson бейзлайном мы строим Scaled-baseline Hawkes — модель с фиксированной персонализированной базой и pooled Хоукс-надстройкой:

$$\lambda_{u,t} = c \cdot \mu_u^{\text{EB}} \cdot b_t + \sum_{j,m} \alpha_{j,m} \cdot z_{u,j,m,t},$$

где c — глобальный масштаб базы (один скаляр на всю когорту), а $z_{u,j,m,t}$ — Hawkes-state с полураспадом m для признака j . В основной реализации мы используем 5 признаков $\times$ 2 полураспада (1, 3) дня, что даёт 10 параметров α и один скаляр c — итого 11 обучаемых параметров.

Обучение двухступенчатое:

1. **ЕВ-шаг** (наследуется из бейзлайна): из Personalized GP получены μ_u^{EB} , (α, β) и b_t . На этом шаге per-user множители зафиксированы.
2. **Хоукс-шаг**: численная оптимизация (c, α) методом L-BFGS-B с ограничением $\alpha \geq 0$. Loss — Пуассоновский негативный log-likelihood с мягкой L2-регуляризацией:

$$\mathcal{L}(c, \alpha) = -\log p(y \mid c\lambda_{\text{base}} + X\alpha) + \lambda_\alpha \|\alpha\|_2^2 + \lambda_c (c - 1)^2.$$

На главном 207d-train Scaled Hawkes даёт test NLL 0.3958 против 0.4096 у Pers GP — улучшение -0.014 в NLL. Обученные параметры: $\hat{c} \approx 0.83$, $\|\alpha\|_2 \approx 0.016$. Тот факт, что $c < 1$, говорит о том, что оптимизатор слегка уменьшает фоновую часть, компенсируя её Хоукс-надстройкой. Это первый намёк на то, что между $c \cdot \mu_u$ и Hawkes-states существует трейдофф, к которому мы вернёмся в главе III.

Обученная матрица $\alpha[\text{feature} \times \text{half-life}]$ визуализируется на рис. 7. Наибольшее значение по абсолютной величине получает коэффициент при self-history `to_ord` (`hl=3`); однако сам канал `to_ord` крайне разреженный, и даже при таком весе вклад этого ядра в общую интенсивность остаётся небольшим. Положительные α для `searches` и `to_cart` на масштабе 1 день соответствуют короткой воронке `search` $\rightarrow$ `cart` $\rightarrow$ `order` в

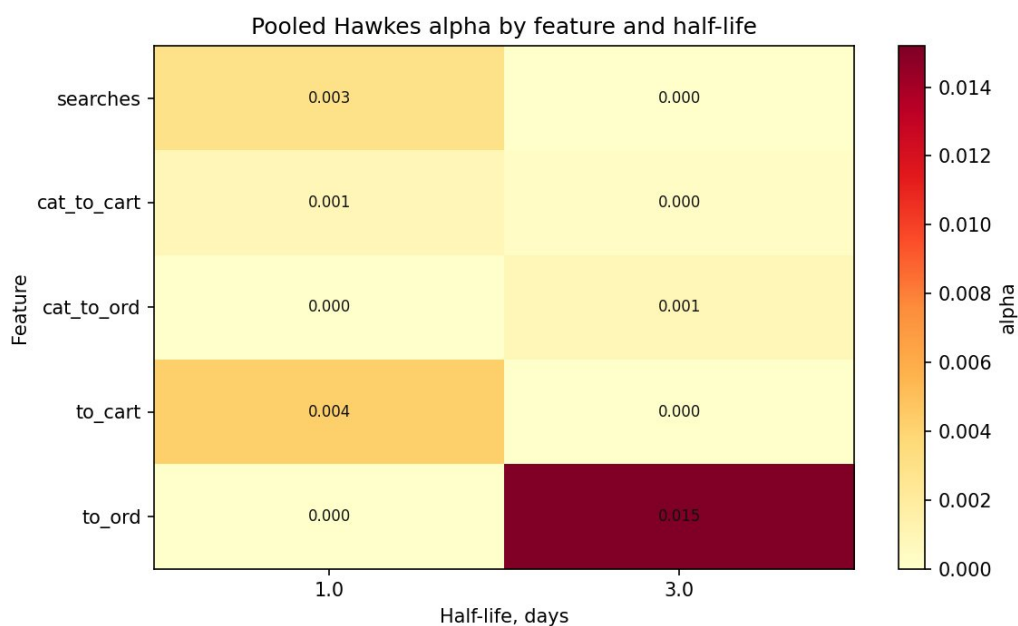


Рис. 7: Тепловая карта Хоукс-коэффициентов: feature $\times$ half-life (Scaled Hawkes)

пределах одной-двух сессий пользователя. Устойчивость коэффициентов мы отдельно исследуем в Главе III.

Анализ per-user delta-LL (рис. 8) показывает, что Хоукс-настройка даёт выигрыш у 63.4% пользователей. В отличие от перехода к персонализации, выигрыш распределён более равномерно по всем бакетам активности (54..80% пользователей выигрывают в каждом бакете).

Среднее / медиана delta-LL: +0.71 / +0.16 на пользователя. Это значит, что среднее улучшение скромное, но устойчивое — нет хвоста пользователей, на которых Hawkes резко проигрывает бейзлайну. Это критичное практическое свойство: модель надёжна на всей когорте.

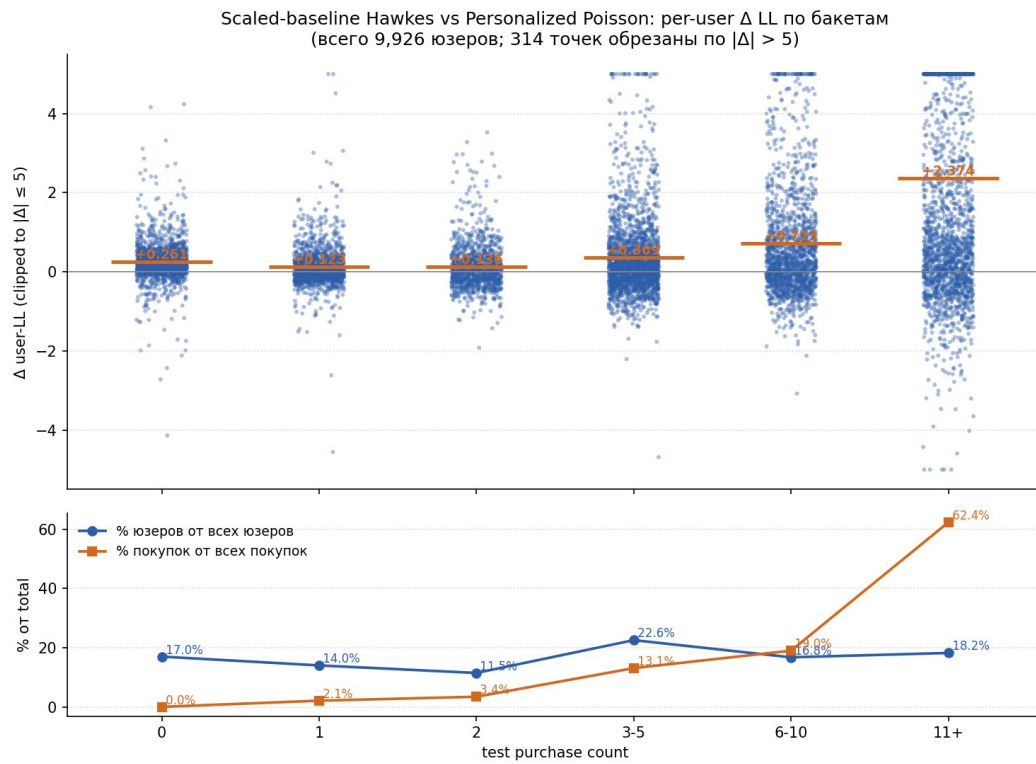


Рис. 8: Per-user delta-LL: Personalized $\rightarrow$ Scaled Hawkes

Регуляризация и выбор полураспадов. Для модели на 11 параметрах с $\sim 2\text{М}$ строк train сетка регуляризации $(\lambda_\alpha, \lambda_c) \in \{0, 0.01, 0.1, 1, 10, 100\}^2$ показывает, что результат не зависит от регуляризации в широком диапазоне: размах test NLL по всей сетке составляет менее 0.0005. В целом это согласуется с ожиданиями: у нас много данных, и всего 11 параметров.

Выбор полураспадов — отдельный методический вопрос. Мы тестировали значения $\tau \in \{0.5, 1, 2, 3, 5, 7\}$ дня в комбинации с разным числом параллельных ядер. Главные наблюдения:

- Один полураспад $\tau = 1$ день даёт самый простой и быстрый фит, однако проигрывает набору полураспадов (1, 3) примерно 700 LL на тестовой выборке.
- Комбинация (1, 3) дня даёт оптимум по test NLL и сохраняет интерпретируемость двух разделённых масштабов памяти: «немедленный» (сегодня $\rightarrow$ завтра) и «среднесрочный» (3-дневная сессия).
- Добавление дополнительных полураспадов почти не увеличивает скор на тесте. При этом веса ядер становятся менее стабильными: два длинных полураспада сильно скоррелированы, и линейная модель распределяет веса между ними фактически случайно.

В дальнейшем в работе используется конфигурация (1, 3) дня для всех Хоукс-моделей в частях 1, 2.

4.4 Joint Hawkes как альтернативная параметризация

Рядом со Scaled-baseline Hawkes мы вводим вторую Хоукс-параметризацию — Joint Hawkes, где per-user множители λ_u и коэффициенты α обучаются совместно напрямую через MLE с явной L2-регуляризацией:

$$\lambda_{u,t} = \lambda_u \cdot b_t + \alpha^\top z_{u,t}, \quad \mathcal{L} = \text{NLL}(y, \lambda) + \lambda_{\ell_2} \sum_u (\lambda_u - 1)^2 + \lambda_\alpha \|\alpha\|_2^2.$$

Главные отличия от Scaled-baseline:

- Одностадийное обучение — без отдельного EB-шага. Empirical-Bayes prior на μ_u заменён явным L2-prior'ом $(\lambda_u - 1)^2$, и оптимизатор учитывает его одновременно с Хоукс-частью.
- λ_u обучается напрямую, а не наследуется зафиксированным из бейзлайна. Это даёт оптимизатору свободу «перекинуть» массу интенсивности между λ_u и Хоукс-вкладом — Scaled-baseline такой возможности не имеет, потому что μ_u^{EB} зафиксирован.

Обучаем всё так же, численно с помощью L-BFGS-B. Сходимость получается довольно быстрой.

На главном 207d-train Joint Hawkes даёт почти то же качество, что и Scaled-baseline (test NLL 0.3951 vs 0.3958) — по NLL модели на полном объёме данных практически неотличимы. Но отмечу, что структура регуляризации у них принципиально разная. Одинаковый скор не означает автоматически, что мы сошлись в одинаковые оптимумы. В дальнейшем будем исследовать этот момент подробнее.

4.5 Бустинг

Чтобы сравнение Хоукс-моделей с бустингом было объективным, бустинг мы тоже старались максимально вытянуть. Основной рычаг улучшения оказался со стороны конструирования признаков: из 5 исходных поведенческих счётчиков (`searches`, `cat_to_cart`, `cat_to_ord`, `to_cart`, `to_ord`) и нескольких производных каналов мы построили 141 признак, покрывающую историю, тренды, ЕМА, recency и funnel-ratio. Конкретный набор признаков можно сгруппировать:

- **Календарные (3):** `dow`, `days_seen`, `week_idx`.
- **History по каждому source-каналу** ($14 \times 8 = 112$): `*_yesterday`, `*_sum3`, `*_sum7`, `*_mean7`, `*_active_days7`, `*_exp3`, `*_last_active`, `*_days_since_last_active`.
- **Агрегаты по заказам и активности:** `orders_hist_mean`, `carts_hist_mean`, `any_activity_sum7`, `any_activity_mean7` и т.п.
- **ЕМА:** `to_ord_ewm7`, `to_ord_ewm28`, `to_ord_ewm_gap_7_28`, `to_cart_ewm7`, `to_cart_ewm28`, `any_activity_ewm7`, `any_activity_ewm28`.
- **Recency и funnel-ratio:** `days_since_last_*`, `ord_per_cart_7`, `search_to_cart_rate_7`, `search_to_ord_rate_7`, `cat_to_cart_rate_7`, `cat_to_ord_rate_7`.

В качестве модели используем `HistGradientBoostingRegressor` с Poisson loss. Финальные гиперпараметры: `max_iter=200`, `max_depth=5`, `learning_rate=0.05`, `min_samples_leaf=40`, early stopping по held-out fraction. Со стороны гиперпараметров мы провели несколько точечных экспериментов (вариации глубины, learning rate, числа итераций), но заметного отклика по NLL они не дали — в итоге зафиксировали усреднённо-разумную конфигурацию без агрессивного тюнинга.

Test NLL бустинга — 0.3881, что лучше Scaled-baseline Hawkes (0.3958). Выигрыш составляет $-0.007.. - 0.008$ в NLL. Это означает две вещи. Во-первых, GBDT действительно ловит дополнительный сигнал, которого нет в Hawkes — вероятно, нелинейные взаимодействия между признаками. Во-вторых, его абсолютное улучшение относительно Hawkes мало по сравнению с разрывом между Hawkes и теоретическим saturated floor

(0.0862): даже сильная бустинг-модель со 141 признаком закрывает лишь небольшую долю этого разрыва. Мы интерпретируем это как косвенный признак того, что большую часть оставшегося разрыва составляет шум в данных — сигнал, который не извлекается ни линейной аддитивной формой Hawkes, ни нелинейной формой GBDT на имеющихся признаках.

В работе GBDT отмечается отдельным цветом — как референсная точка, дающая верхнюю планку качества. Цель — оценить, на каком расстоянии от этой планки находятся интерпретируемые Хоукс-модели.

4.6 Итоговая лестница моделей

Соберём все рассмотренные модели в единую сводную картину. Получается лестница из шести вероятностных моделей плюс GBDT и теоретический saturated floor:

1. Global Poisson — единая константа $\bar{\lambda}$ на весь датасет.
2. Rolling Poisson — скользящее среднее дневного уровня, без персонализации.
3. Rolling Seasonal Poisson — с поправкой на день недели.
4. Personalized Gamma-Poisson — персонализация множителем μ_u на предыдущую модель через Байесовский вывод.
5. Scaled-baseline Hawkes — Personalized GP с Хоукс-надстройкой.
6. Joint Hawkes — совместная оптимизация λ_u и α напрямую через MLE с L2-prior’ом (см. подсекцию выше).
7. GBDT — контрольная модель на расширенном синтетическом наборе признаков (см. предыдущую подсекцию).

Каждый следующий уровень включает в себя структуру предыдущего и добавляет один концептуальный шаг.

Самый крупный единичный шаг ожидаемо приходится на персонализацию (Gamma-Poisson, -0.0480 в NLL): без неё модель неизбежно усредняет активных и неактивных пользователей. Все четыре «не персонализированных» бейзлайна почти не отличаются друг от друга — разброс между пользователями кратно превышает изменчивость во времени.

Содержательно более интересен следующий шаг: Хоукс-надстройка даёт значимый прирост поверх и без того сильного персонализированного бейзлайна (-0.0138 в NLL). Joint Hawkes отличается от Scaled-baseline всего на -0.0007 — на уровне численного шума, поэтому говорить о выигрыше одной параметризации над другой на полном train не приходится.

GBDT, как и положено сильной табличной модели на 141 признаке, занимает верхнюю планку (-0.0070 в NLL поверх Joint Hawkes), но отрыв от Хоукс-моделей минимален. Если посчитать, какую долю зазора между Personalized GP и GBDT закрывают Хоукс-модели, получится $\sim 64\%$ для Scaled-baseline и $\sim 67\%$ для Joint Hawkes — то есть Хоукс-надстройка одна снимает около двух третей всего извлекаемого бустингом сигнала, при существенно более простой и интерпретируемой структуре.

Наглядно лестница по per-observation NLL представлена на рис. 9.

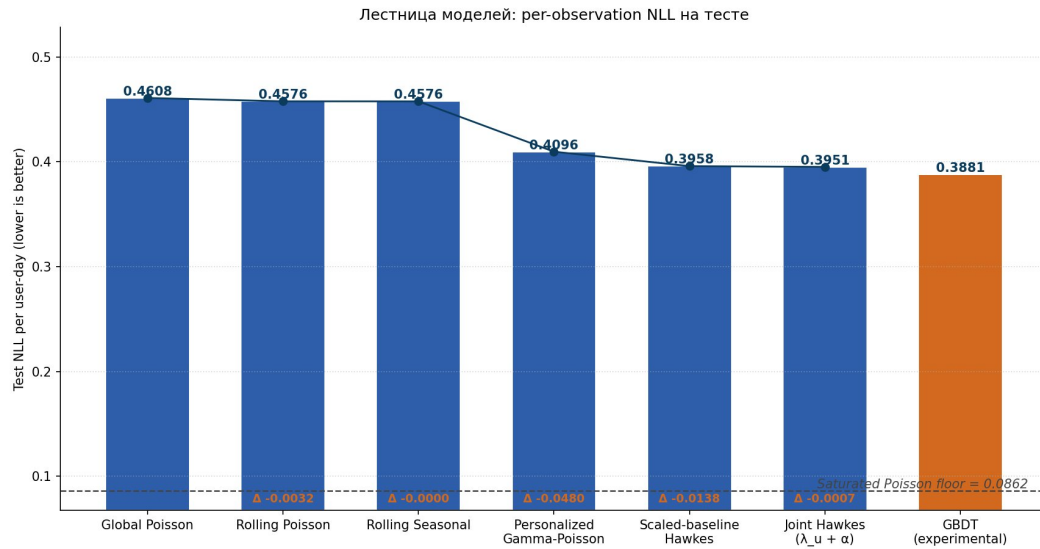


Рис. 9: Per-observation NLL: лестница моделей

Кратко главные наблюдения:

- Крупный шаг по NLL даёт переход к персонализации.
- Хоукс-надстройка добавляет ещё стабильный шаг порядка 0.014 в NLL.
- Между Scaled-baseline и Joint Hawkes разница ничтожна (0.0007 в NLL).
- GBDT обходит все вероятностные модели, но с менее интерпретируемой структурой и с очень небольшим отрывом.

Доля закрытого зазора между Global Poisson и saturated floor'ом, накопленная к Joint Hawkes, составляет $\sim 18\%$; до GBDT — $\sim 19\%$. Тот факт, что даже сильная бустинг-модель закрывает лишь немногим больше Joint Hawkes, мы интерпретируем как косвенное указание на то, что значительную часть оставшихся $\sim 80\%$ составляет шум в данных — сигнал, не извлекаемый ни линейной аддитивной формой Hawkes, ни нелинейной формой GBDT на текущих признаках.

Численная сводка по всем ступеням:

Вторичные метрики (абсолютный LL и RMSE) повторяют ту же лестницу. В LL: переход к персонализации даёт скачок порядка +24K, Хоукс-надстройка — ещё +7K, GBDT улучшает Joint Hawkes ещё на +3.6K. RMSE монотонно убывает от бейзлайна к

Ступень	Модель	Test LL	NLL/n	Δ vs предыдущая (NLL)
1	Global Poisson	-236458	0.4608	—
2	Rolling Poisson	-234805	0.4576	-0.0032
3	Rolling Seasonal	-234782	0.4576	-0.0000
4	Personalized Gamma-Poisson	-210167	0.4096	-0.0480
5	Scaled-baseline Hawkes	-203093	0.3958	-0.0138
6	Joint Hawkes ($\lambda_u + \alpha$)	-202721	0.3951	-0.0007
7	GBDT (experimental)	-199154	0.3881	-0.0070
-	Saturated Poisson floor	-44216	0.0862	—

Hawkes (0.655 $\rightarrow$ 0.627); GBDT практически совпадает с Joint Hawkes по RMSE (0.6297 vs 0.6291). Сводный график — рис. 10.

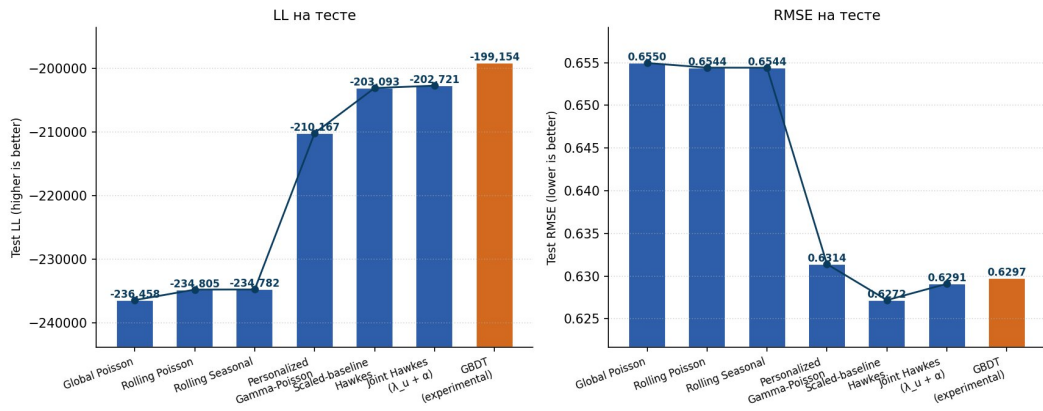


Рис. 10: Test LL и RMSE по моделям

4.7 Декомпозиция интенсивности: Scaled-baseline vs Joint Hawkes

Главный test NLL у Scaled-baseline Hawkes и Joint Hawkes практически совпадает. Это естественно поднимает вопрос: учат ли эти модели одно и то же разложение интенсивности на бейзлайн-часть и Хоукс-надстройку, или приходят к разным внутренним структурам, лишь случайно дающим одинаковый общий скор?

Чтобы это проверить, строим облако точек обученных per-user множителей: m_u из Scaled-baseline Hawkes (где $m_u = c \cdot \mu_u^{EB}$) против m_u из Joint Hawkes (где $m_u = \lambda_u$ напрямую).

Структурные эффекты:

- **Средние значения** (красный крест): $m_{staged} \approx 0.823$, $m_{joint} \approx 0.649$. Joint в среднем занижает множитель сильнее, компенсируя это примерно в 2 раза большим $\|\alpha\|_2$ (0.0324 vs 0.0161).
- **Корреляция** $\text{corr}(m_{staged}, m_{joint}) = 0.85$ — высокая, но не единица: модели договариваются о том, кто более активный, но конкретные значения отлича-

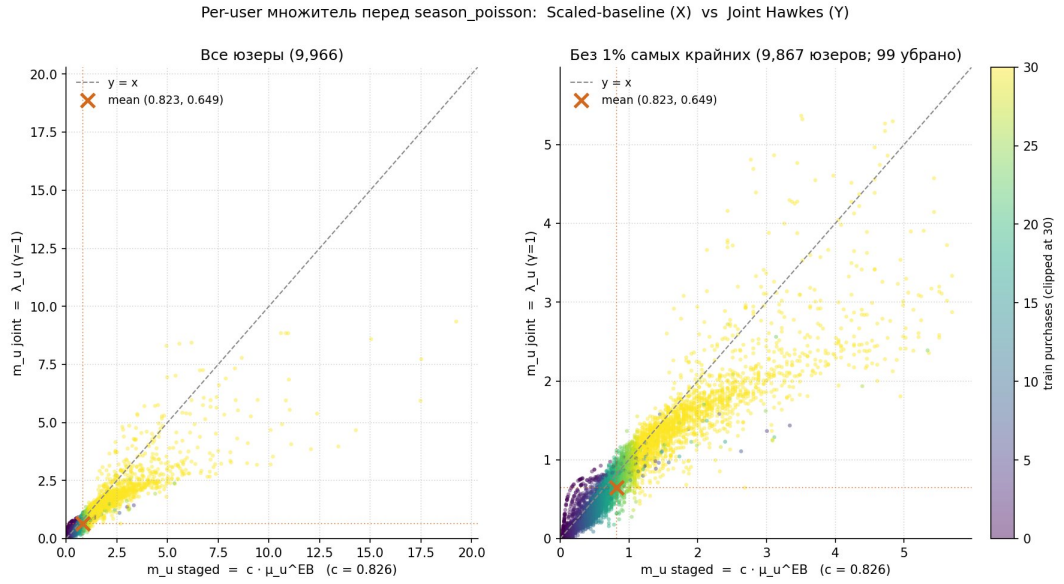


Рис. 11: Per-user m_u : Scaled-baseline vs Joint Hawkes

ются.

- **Активные пользователи** (правый хвост) ниже диагонали: joint-фит снижает per-user множитель, оставляя место большой Хоукс-надстройке.

Несмотря на эти разные декомпозиции, итоговый test NLL обеих моделей практически совпадает. Это и есть тот самый c — α трейдофф: разные оценки (m_u, α) дают почти одинаковую общую интенсивность. На уровне population-NLL модели неотличимы; на уровне внутренних параметров — существенно разные. Это наблюдение играет центральную роль в главе III, где мы разворачиваем его в полноценный анализ identifiability через profile likelihood.

5 Глава II. Поведение моделей на коротких train-окнах

Все результаты Главы I получены на 207-дневном train. Возникает естественный вопрос: насколько эти выводы устойчивы к выбору train-окна? В индустриальных задачах часто доступны только короткие истории (14..30 дней — например, в задачах быстрого онбординга новых сегментов или анализа А/В-эксперимента), и важно понять, какая параметризация Hawkes выживает на таких окнах.

5.1 Альтернативное train-test разбиение

Первый эксперимент — поблочная кросс-валидация с непересекающимися 21-дневными блоками. Анализируемый период 2025-01-15..2025-09-30 бьётся на 13 блоков. В каждом блоке первые 14 дней используются как train, последние 7 — как test. На каждом блоке независимо обучаются все шесть моделей лестницы плюс GBDT.

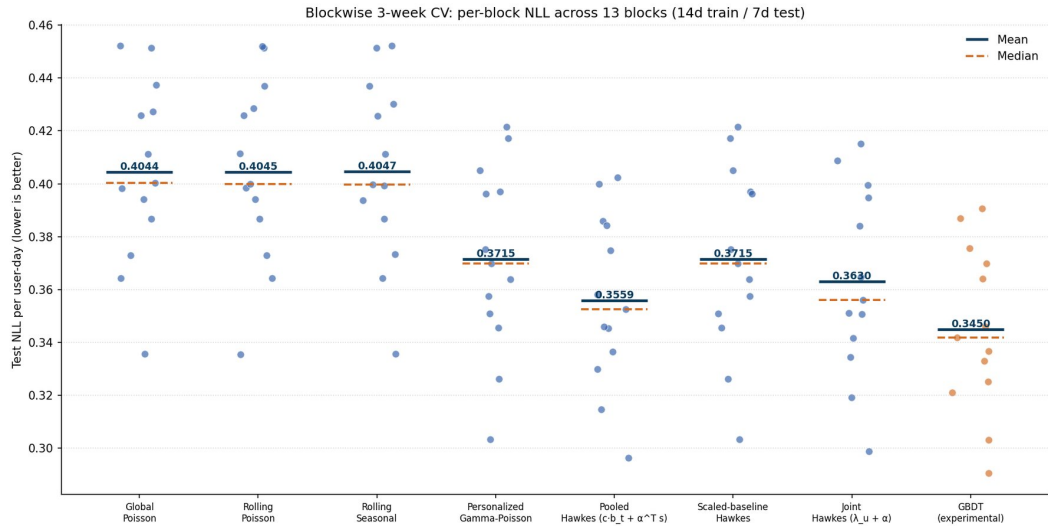


Рис. 12: Поблочная CV: per-block NLL

На коротких train (14d) картина меняется по сравнению с главным окном. Несколько ключевых наблюдений:

- **Все «не персонализированные» бейзлайны** (Global, Rolling, Rolling Seasonal) дают практически одинаковое среднее NLL ~ 0.404 . Это и ожидаемо: скользящее среднее на 14 днях не сильно отличается от глобального среднего, а недельная сезонность на 14 днях оценивается шумно.
- **Personalized Gamma-Poisson** даёт среднее NLL ~ 0.372 — персонализация через Empirical Bayes остаётся устойчивым шагом и здесь. Абсолютные значения у всех моделей ниже, чем на главном train (там Pers GP давал ~ 0.41), — это связано в первую очередь с тем, что test-окно тут всего 7 дней, а на главном протоколе — 52, поэтому среднее по более короткому test шумнее и в среднем ниже.

- **Scaled-baseline Hawkes вырождается** на этих окнах: Хоукс-веса прижимаются к нулю, и модель фактически становится копией Personalized GP — отсюда и совпадение среднего NLL (~ 0.372). Подробно разбираем причину в следующей подсекции.
- **Joint Hawkes** даёт среднее NLL ~ 0.363 — лучший среди вероятностных моделей, и заметно лучше Pers GP. Хоукс-надстройка здесь не вырождается. Тезис о том, что регуляризация (в том числе структурная) Хоукс-ядер влияет на итоговые веса, подтверждается.
- **GBDT** остаётся лидером по абсолюту со средним NLL ~ 0.345 .

Дисперсия результатов по блокам велика у всех моделей — это нормально для шумной разреженной задачи на коротких окнах: разные блоки попадают на разные участки активности когорты, и точечное значение NLL ощутимо плавает даже у одной и той же модели.

5.2 Вырождение Scaled-baseline Hawkes

Эффект вырождения Scaled-baseline Hawkes на 14d train особенно поучителен. Если посмотреть на обученные параметры на каждом блоке, видна следующая картина:

- $\hat{c} \approx 1.00$ (зажат к prior'у $c = 1$);
- $\|\alpha\|_2 \approx 0$ (все Хоукс-веса прижаты к нулю).

То есть модель полностью теряет Хоукс-надстройку и фактически сводится к Personalized GP. На уровне предсказательного качества это и подтверждается: их test NLL практически совпадают.

Почему так получается. На 14 днях у большинства пользователей всего 0-2 события целевого канала, и EB-prior тянет μ_u^{EB} к среднему по когорте. С одной стороны, Personalized GP уже неплохо подгоняет такие редкие события одним per-user уровнем — оптимизатору почти нечего «доулучшать» Хоукс-надстройкой. С другой стороны, событий, на которых Хоукс-ядро вообще могло бы себя показать, тоже мало: $z_{u,j,t}$ редко отличается от нуля, и его в среднем не отличить от собственно бейзлайна. В итоге оптимизатор предпочитает оставить ровно одну компоненту — бейзлайн, тем более что его prior подталкивает $c \rightarrow 1$.

Вырождение мотивирует переход к другой форме обучения, которая бы не позволяла Хоукс-надстройке схлопнуться к нулю.

5.3 Joint Hawkes на коротких окнах

Joint Hawkes (определение и обучение разобраны в Главе I) построен так, что вместо двухступенчатого EB-prior'a на μ_u использует явную L2-регуляризацию per-user множителей $(\lambda_u - 1)^2$ напрямую при совместной оптимизации с α . Это отличие, малозаметное

на полном train (test NLL практически одинаковый со Scaled-baseline), оказывается решающим на коротких окнах.

Активный L2-prior удерживает λ_u в окрестности единицы: оптимизатор не может «зажать» все λ_u в ноль и переложить весь сигнал на Хоукс-часть (или наоборот). На 14d-train Joint Hawkes **не вырождается**: $\|\alpha\|_2 \approx 0.04..0.05$ на всех n от 15 до 200 дней. То есть Хоукс-надстройка остаётся ненулевой и информативной даже там, где у Scaled-baseline она схлопывается до нуля.

По умолчанию используем $\lambda_{\ell_2} = 1$ как «активный prior». Чувствительность результата к этой константе подробно разбираем в Главе III; здесь она фиксирована.

5.4 Анализ результатов в зависимости от длины train-test разбиения

Систематический анализ — перебор по длине train-окна $n \in \{15, 21, 27, 36, 48, 63, 84, 114, 153, 198\}$ дней. На каждом значении n берётся m случайных интервалов из анализируемого периода (с фиксированным seed), и для каждого интервала строится разбиение $2n/3$ train + $n/3$ test. На каждом запуске обучаются все семь моделей. Это позволяет получить distribution test NLL для каждой пары (модель, n) и оценить как среднее, так и дисперсию.

Сводный график среднего test NLL по моделям от n — рис. 13.

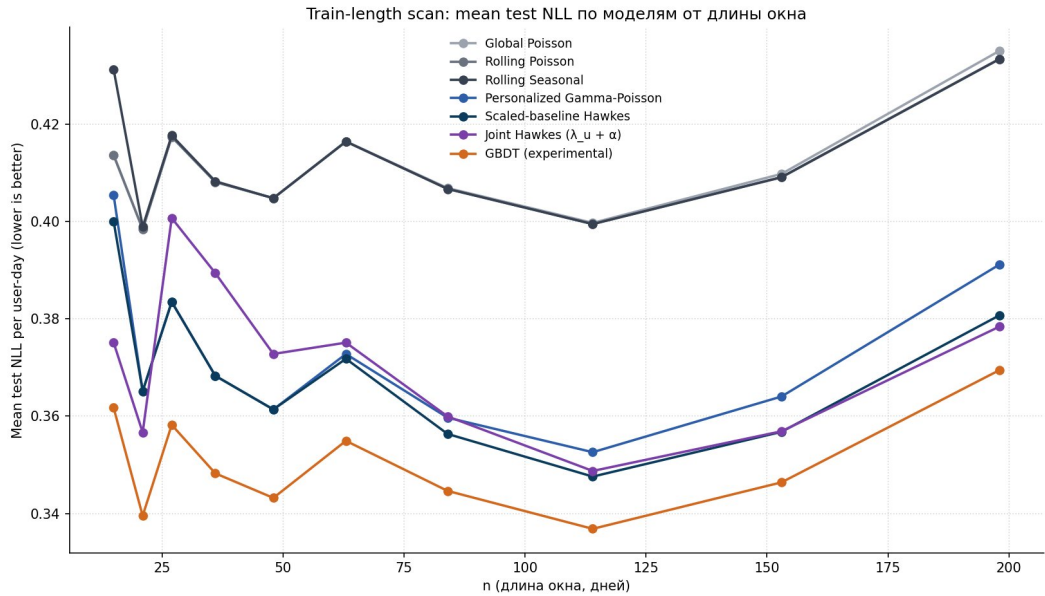


Рис. 13: Среднее NLL по моделям от длины окна

Видно чёткое разделение по зонам n :

- На $n \leq 21$ Joint Hawkes лидирует с заметным отрывом ($\sim 0.02..0.05$ в NLL): Scaled-baseline в этой зоне вырождается, а Joint с активной L2-регуляризацией удерживает Хоукс-сигнал.

- На $n \in [27, 60]$ ситуация переворачивается: Joint Hawkes становится хуже Scaled-baseline и Personalized GP. Похоже, что выставленная по умолчанию сила L2-регуляризации ($\lambda_{\ell_2} = 1$), удерживающая модель от вырождения на самых коротких окнах, на этих промежуточных длинах уже излишне ограничивает λ_u и Хоукс-веса.
- На $n \geq 100$ Scaled-baseline и Joint Hawkes по test NLL сравниваются и оба обходят Personalized GP примерно на ~ 0.02 в NLL. Важно подчеркнуть: совпадение здесь только по *итоговому* NLL — сами оптимумы (λ_u, α) у двух моделей различны (см. подсекцию 4.7 про декомпозицию интенсивности).

GBDT остаётся лучшим в абсолюте на всех n , но разрыв уменьшается с ростом n — на длинных train вероятностные модели догоняют бустинг. На $n = 198$ день разрыв Joint Hawkes vs GBDT составляет ~ 0.009 в NLL против ~ 0.014 на $n = 15$.

Поведение нормы Хоукс-коэффициентов и среднего рег-*user* множителя от n — рис. 14 и 15.

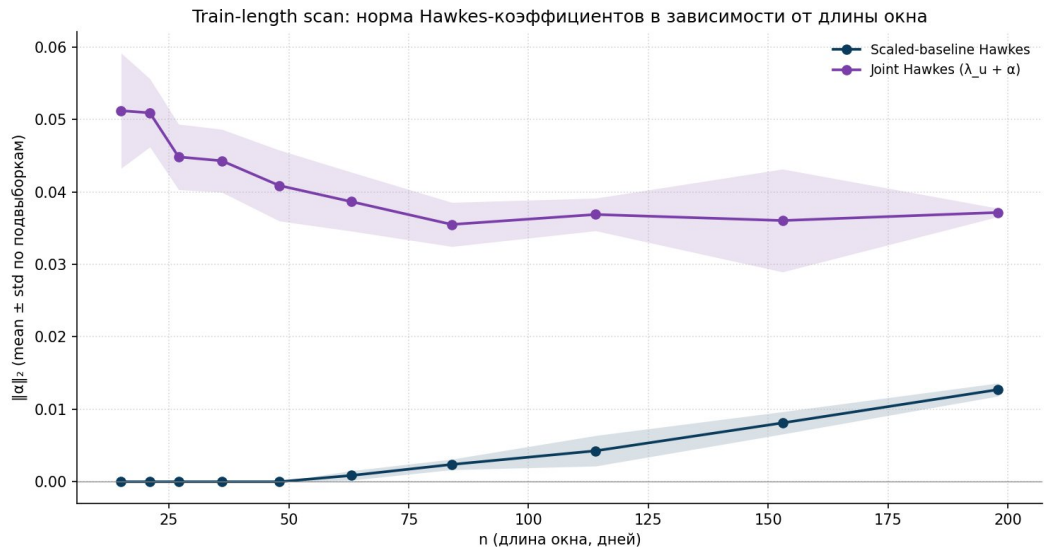


Рис. 14: Норма $\|\alpha\|_2$ по моделям от длины окна

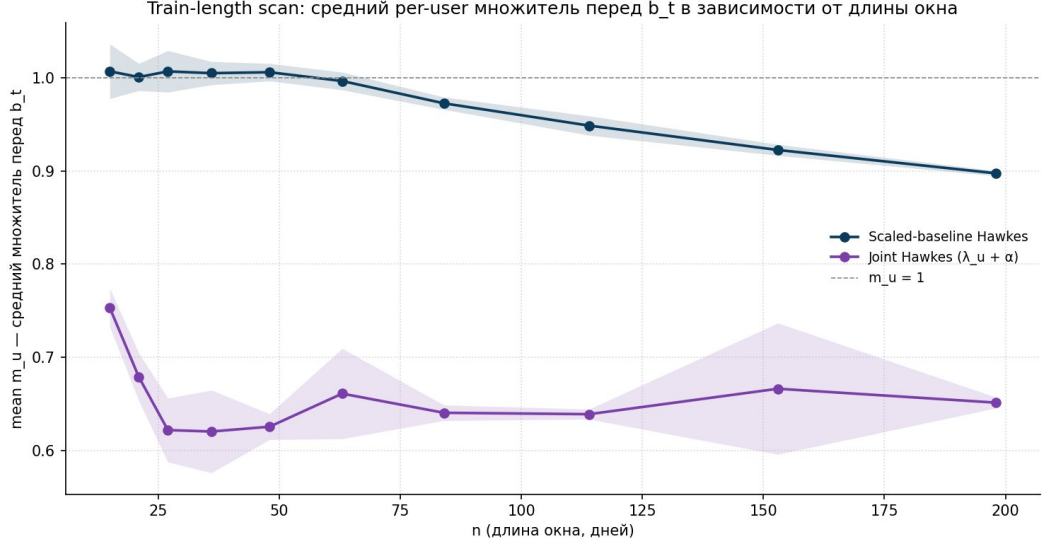


Рис. 15: Средний per-user множитель от длины окна

Здесь хорошо видно вырождение: для Scaled-baseline $\|\alpha\| = 0$ на всех $n \leq 50$ (плоское ноль), и только с $n \approx 80$ норма начинает расти. На $n = 198$ норма достигает ~ 0.012 , что близко к значению ~ 0.016 , полученному на полном 207d-train. У Joint Hawkes $\|\alpha\|$ стабилен ($\sim 0.04..0.05$) с самого $n = 15$, медленно убывая к ~ 0.037 на больших n . Среднее значение per-user множителя λ_u у Joint Hawkes держится $\sim 0.65..0.75$ независимо от n , что согласуется с активной L2-регуляризацией к единице.

Кроме того видно, что у Scaled-baseline норма $\|\alpha\|$ стабильно растёт с увеличением числа дней обучения. Логично предположить, что это происходит потому, что на больших выборках слабый Хоукс-сигнал получается лучше различать в шумных данных. Вполне вероятно, что на ещё более длинных данных Хоукс-веса двух параметризаций сошлись бы к одному значению. Но на нашем диапазоне n даже на 200 днях разница между ними остаётся значительной.

Главный вывод этой части: Хоукс-сигнал в данных действительно присутствует, но он слабый, и форма регуляризации существенно меняет картину. При структурной регуляризации через двухступенчатый EB-fit (Scaled-baseline) на коротких train Хоукс-сигнал не отделяется от бейзлайна и поглощается переобученным per-user множителем — модель вырождается фактически до Personalized GP. Scaled-baseline за счёт этого нигде заметно не проигрывает Pers GP — в зоне вырождения он просто совпадает с ним. При явной L2-регуляризации (Joint Hawkes) Хоукс-надстройка сохраняется на всём диапазоне n , но это не «бесплатно»: на самых коротких окнах ($n \leq 21$) Joint Hawkes выигрывает, тогда как на промежуточных ($n \in [27, 60]$) тот же активный L2-prior уже излишне ограничивает модель, и Joint Hawkes на этих длинах оказывается даже хуже Pers GP. Устойчивое преимущество Хоукс-надстройки над Personalized GP начинается с $n \approx 80$ дней и дальше только накапливается: на более коротких окнах значимого выигрыша ждать не стоит.

6 Глава III. Структура cross-channel Хоукс-матрицы

В первой части мы рассматривали одну целевую переменную — `to_ord` — и вектор α по нескольким source-каналам. В этой части переходим к cross-channel постановке: на каждый из трёх каналов воронки (`searches`, `to_cart`, `to_ord`) подгоняется собственная Хоукс-модель, у которой источниками возбуждения служат все три канала (включая сам себя). Получается интерпретируемая матрица $\alpha \in \mathbb{R}^{3 \times 3}$ пар-канальных взаимодействий — диагональ описывает самовозбуждение каждого канала, внедиагональные элементы — cross-channel влияние.

6.1 Постановка и верификация cross-channel модели

Бейзлайн для каждого из трёх каналов (`searches`, `to_cart`, `to_ord`) строится точно так же, как в Главе I: Personalized Gamma-Poisson с собственным per-user множителем $\mu_u^{\text{EB},k}$ и собственным дневным профилем $b_t^{(k)}$ для каждого целевого канала k . Поверх него натягивается Хоукс-надстройка, источниками возбуждения которой служат все три канала (включая сам себя):

$$\lambda_{u,t}^{(k)} = c_k \cdot \mu_u^{\text{EB},k} \cdot b_t^{(k)} + \sum_{j=1}^3 \alpha_{kj} \cdot z_{u,j,t}.$$

Hawkes-state $z_{u,j,t}$ строятся лишь с одним полураспадом в 1 день. Мы знаем, что метрики из-за этого проседают слабо, зато легче выделяется структура коэффициентов. Оптимизация выполняется отдельно по каждому целевому каналу. Главный α -объект исследования — матрица $\alpha \in \mathbb{R}^{3 \times 3}$ пар-канальных взаимодействий: диагональ описывает самовозбуждение каждого канала, внедиагональные элементы — cross-channel влияние.

Верификация: даёт ли Хоукс-надстройка прирост на каждом канале? Прежде чем разбираться в структуре матрицы α , нужно убедиться, что Хоукс-надстройка вообще полезна для всех трёх каналов (для `to_ord` это уже показано в Главе I, а `searches` и `to_cart` — новые целевые каналы). Сравнение per-channel test NLL у Pers GP, Scaled Hawkes и GBDT приведено на рис. 16.

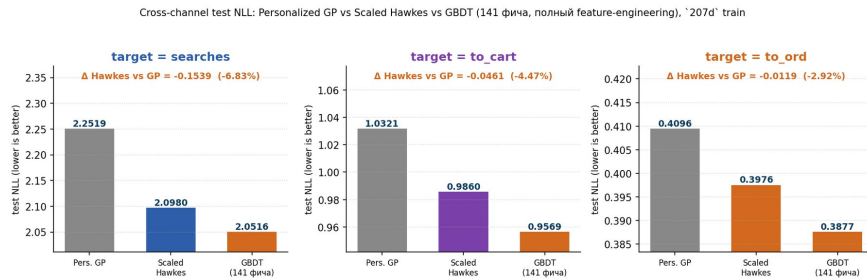


Рис. 16: Per-channel верификация: Pers GP vs Scaled Hawkes vs GBDT

Хоукс-надстройка улучшает Pers GP на всех трёх каналах; GBDT (141 признак

на каждый канал) задаёт верхнюю планку. Интересная закономерность — чем плотнее канал, тем большую долю прироста бустинга поверх Pers GP объясняют сами Хоукс-ядра: на **searches** Hawkes закрывает $\sim 77\%$ gap’а от Pers GP до GBDT, на **to_cart** $\sim 67\%$, на разреженном **to_ord** — $\sim 54\%$. То есть на богатых каналах большая часть извлекаемого сигнала структурно описывается линейной аддитивной Хоукс-формой, а нелинейные взаимодействия играют относительно меньшую роль.

Точечная оценка матрицы α на главном train. Восстановленная матрица для Scaled-baseline Hawkes на 207d-train приведена на рис. 17.

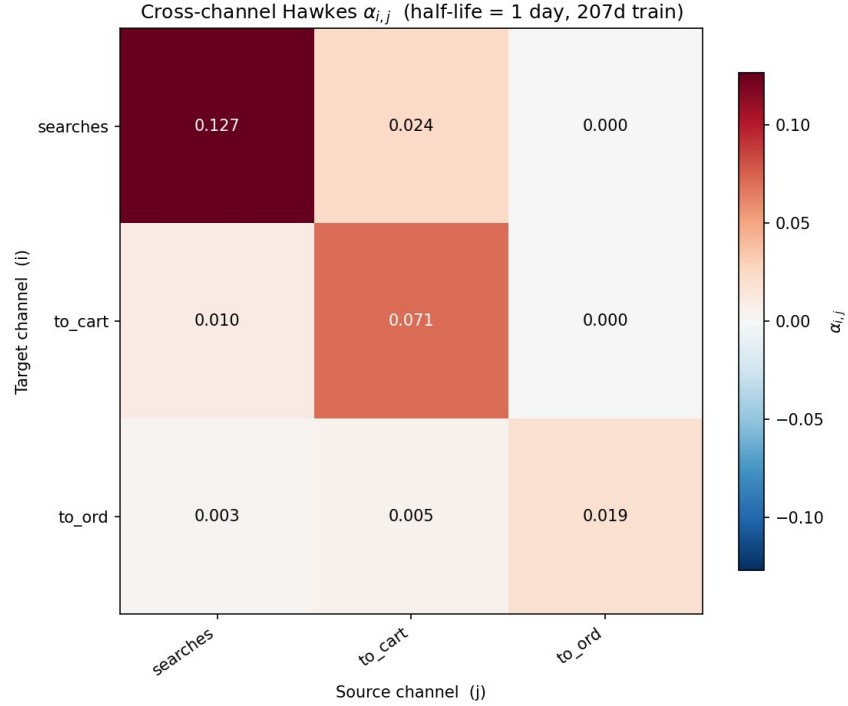


Рис. 17: Cross-channel α -матрица на главном train (Scaled-baseline)

Численные значения:

target ↓ \ source →	searches	to_cart	to_ord
searches	0.1269	0.0240	0
to_cart	0.0103	0.0713	0
to_ord	0.0030	0.0046	0.0194

Визуально матрица показывает осмысленную структуру воронки:

- Верхне-треугольная часть колонки **to_ord** идентично нулевая: краткосрочных Хоукс-эффектов от **to_ord** на **searches** и **to_cart** мы не наблюдаем.
- Внедиагональные funnel-связи **searches** $\rightarrow$ **cart** $\rightarrow$ **order** различимы и положительны: $\alpha[to_{cart} \leftarrow searches] = 0.010$, $\alpha[to_{ord} \leftarrow to_{cart}] = 0.0046$. Это согласуется со структурой воронки: поиск повышает интенсивность добавления в корзину, корзина повышает интенсивность покупки.

- Self- α $\alpha[to_{ord} \leftarrow to_{ord}] = 0.019$ ненулевой, но небольшой — и, как мы увидим в следующих подсекциях, плохо идентифицируется на наших данных.

Здесь нужно сделать важную оговорку. В Главе II мы уже несколько раз видели, что коэффициенты α Хоукс-ядер ощутимо зависят и от длины train-выборки, и от выбора регуляризации. Поэтому значения в таблице выше следует понимать как точечную оценку на одном конкретном train при конкретном способе регуляризации — часть из них меняется на десятки процентов при варьировании окна или λ_{ℓ_2} . Точная интерпретация каждого коэффициента, его чувствительность к регуляризации и доверительные интервалы разбираются в следующих подсекциях.

6.2 Стабильность коэффициентов: train-окно и регуляризация

Из мотивации прошлых глав следует, что стабильность кросс-Хоукс матрицы нужно исследовать по двум осям: длина train-окна и сила/форма регуляризации. В этой подсекции мы сначала смотрим на коэффициенты Scaled-baseline на бутстрапе и при изменении длины окна, потом сравниваем с Joint Hawkes при разных λ_{ℓ_2} , и сопоставляем обе картины.

Бутстрап Scaled-baseline по 10 случайным 100d-окнам показывает заметную нестабильность точечной оценки. Среднее $\pm$ std по окнам:

ячейка	среднее α	std	CV
searches $\leftrightarrow$ searches	0.0798	0.0058	7.2%
searches $\leftrightarrow$ to_cart	0.0168	0.0026	15.5%
to_cart $\leftrightarrow$ searches	0.0060	0.0009	15.6%
to_cart $\leftrightarrow$ to_cart	0.0347	0.0034	9.7%
to_ord $\leftrightarrow$ searches	0.0011	0.0005	47.4%
to_ord $\leftrightarrow$ to_cart	0.0025	0.0009	36.2%
to_ord $\leftrightarrow$ to_ord	0.0016	0.0024	148.3%

Даже при фиксированной длине окна оценка ощутимо плавает: для редких пар CV доходит до 36..47%, а $\alpha[to_{ord} \leftarrow to_{ord}]$ фактически коллапсирует. Если сравнить эти значения с точечной оценкой на главном 207d-train, видно, что все коэффициенты на коротком окне систематически ниже — Scaled-baseline на 100d недообучает всю матрицу, а не только диагональ; диагональные коэффициенты при этом страдают сильнее внедиагональных.

Joint Hawkes на том же 100d-бутстрапе ведёт себя качественно иначе. С активной регуляризацией ($\lambda_{\ell_2} = 1$) он удерживает $\alpha[to_{ord} \leftarrow to_{ord}] = 0.038$ при CV 14.7%; без регуляризации ($\lambda_{\ell_2} = 0$) — снова коллапсирует. Сравнение трёх режимов — рис. 18.

Из рис. 18 видны две вещи. Во-первых, у Joint Hawkes на λ_{ℓ_2} сильнее всего реагируют именно диагональные элементы, а внедиагональные остаются относительно стабильными. Во-вторых, внедиагональные элементы Joint Hawkes на 100d-бутстрапе

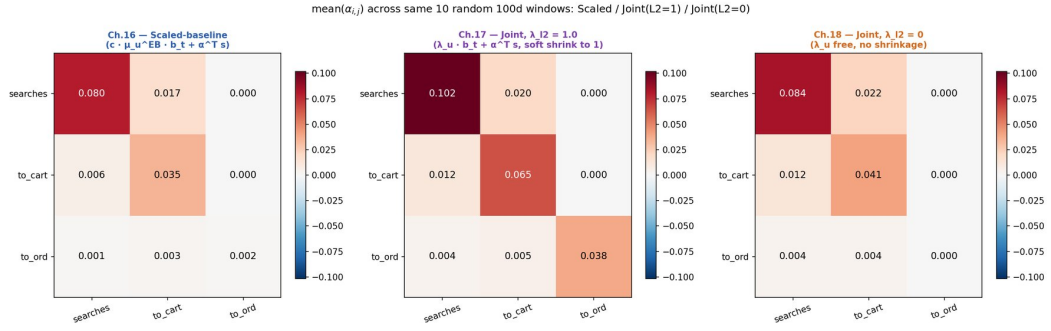


Рис. 18: Сравнение трёх режимов: Scaled vs Joint($\lambda_{\ell_2} = 1$) vs Joint($\lambda_{\ell_2} = 0$)

по величине близки к внедиагональным элементам Scaled-baseline на длинном 207d-train — тогда как сам Scaled-baseline на тех же 100d-окнах даёт заметно меньшие внедиагональные значения.

Связующий вывод. Scaled-baseline на небольших train-выборках систематически недообучает всю матрицу, в том числе внедиагональные; длинный train выводит внедиагональные на устойчивый уровень. Joint Hawkes с активной L2-регуляризацией добивается похожего уровня внедиагональных уже на коротких окнах — за счёт явного prior'a. Диагональные коэффициенты ни в одном из режимов стабильно не восстанавливаются: их точечное значение существенно зависит и от длины окна, и от λ_{ℓ_2} . Этот факт мы формализуем дальше через profile likelihood. Регуляризация может быть либо структурной (EB-fit у Scaled-baseline), либо численной (λ_{ℓ_2} у Joint Hawkes); оба варианта меняют точечные оценки α в одну сторону — перераспределяют массу между бейзлайн-частью и Хоукс-надстройкой.

6.3 Влияние силы регуляризации на матрицу Joint Hawkes

Чтобы понять, как именно регуляризация перераспределяет внутреннюю структуру модели, проведён перебор по $\lambda_{\ell_2} \in \{0, 0.01, 0.05, 0.1, 0.3, 0.5, 1, 2, 5\}$ на главном 207d-train для каждого из 3 целевых каналов. Разложение интенсивности на бейзлайн-часть $\lambda_u \cdot b_t$ и Хоукс-часть $\alpha^\top z$ показано на рис. 19.

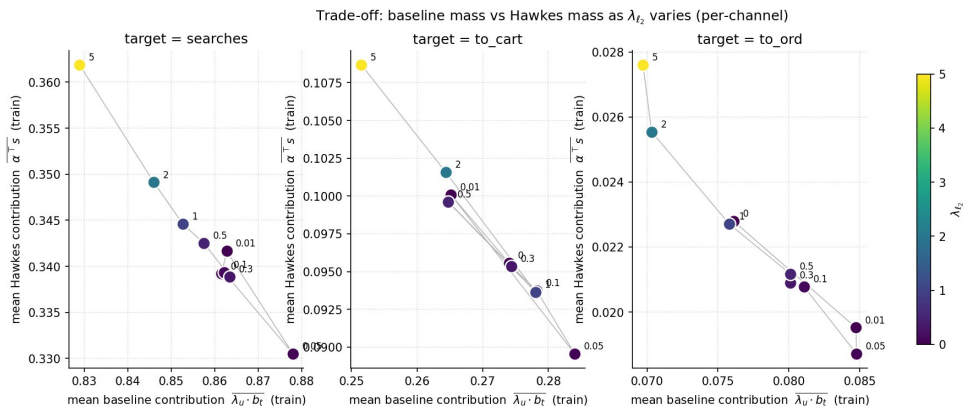


Рис. 19: Trade-off: масса бейзлайна vs Hawkes от λ_{ℓ_2}

С ростом λ_{ℓ_2} оптимизатор зажимает λ_u к 1, и масса перетекает из бейзлайн-части в Хоукс-надстройку — на всех 3 каналах, сильнее всего на разреженном `to_ord`. То есть α выступает компенсаторной переменной по отношению к λ_u . При этом test NLL почти не меняется на всей сетке (рис. 20): размах ≤ 0.01 на всех каналах. Разные λ_{ℓ_2} дают разные внутренние разложения (λ_u, α) , но почти одинаковое предсказание интенсивности на test — модель сверх-параметризована на уровне параметров, но предсказательно устойчива.

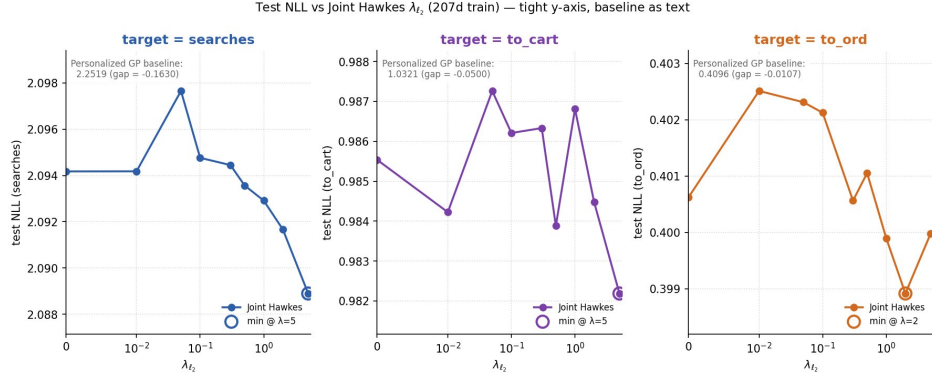


Рис. 20: Test NLL от λ_{ℓ_2}

Разрешение α по λ_{ℓ_2} — рис. 21. Внедиагональные funnel-связи устойчивы ($\pm 2..10\%$), $\alpha[\text{searches} \leftarrow \text{searches}]$ почти не реагирует ($\pm 4\%$): канал богат событиями, self-сигнал отделяется от бейзлайна устойчиво. $\alpha[\text{to_cart} \leftarrow \text{to_cart}]$ меняется умеренно ($\pm 13\%$), а $\alpha[\text{to_ord} \leftarrow \text{to_ord}]$ — сильно ($+82\%$, $0.030 \rightarrow 0.055$). Чем разреженнее целевой канал, тем сильнее зависимость диагонального коэффициента от регуляризации; этот вывод формализуем дальше через profile likelihood.

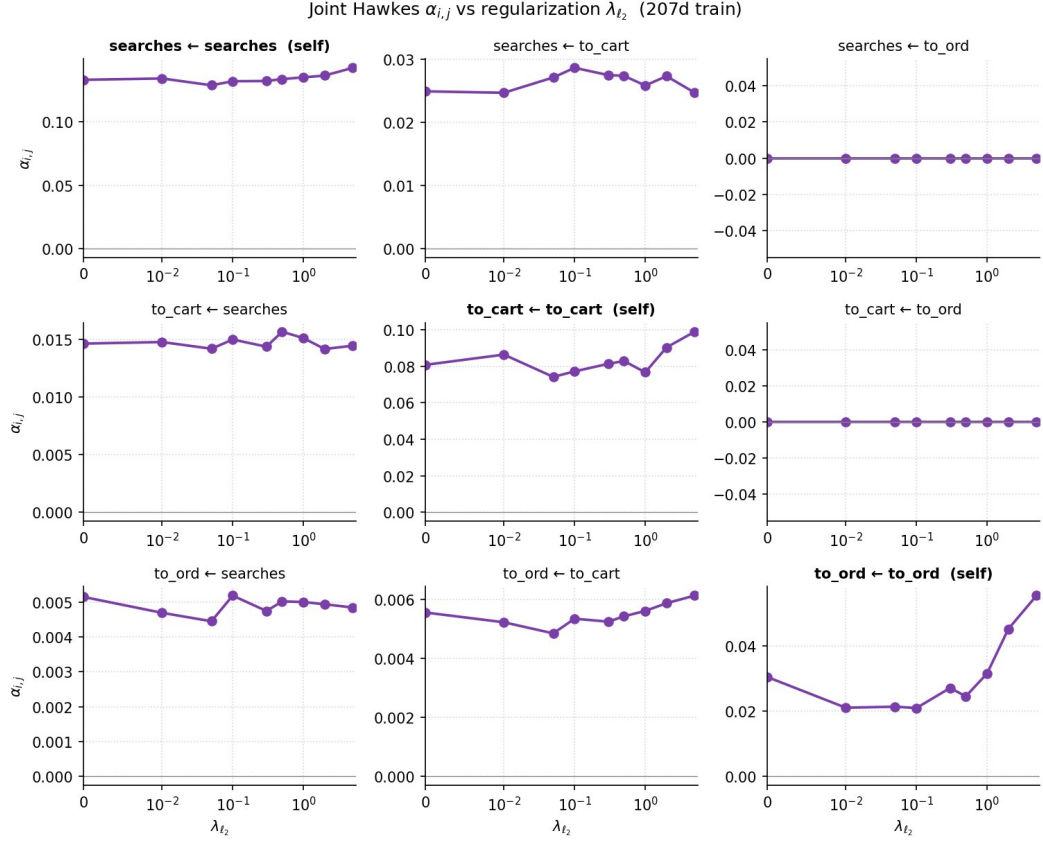


Рис. 21: Зависимость α от λ_{ℓ_2} — 3×3 сетка

6.4 Идентифицируемость коэффициентов: диагональ против внедиагональных

Мы хотим внимательнее рассмотреть, как устроен ландшафт лосса вокруг обученной модели. Для этого попробуем по очереди фиксировать каждый Хоукс-коэффициент на разных значениях, переобучать все остальные параметры модели и смотреть, какого NLL удастся при этом добиться. Если профиль NLL получается узкий и с выраженным минимумом — значит, данные действительно «выбирают» одно значение коэффициента; если плоский — значит, целая полоса значений почти одинаково хороша, и точечная оценка несёт мало информации. Из таких профилей естественно строить границы, в которых Хоукс-коэффициенты можно варьировать без заметной просадки по NLL — это и есть метод *profile likelihood*.

Сам эксперимент устроен так. Для каждой пары канал-источник (k, j) :

1. Обучаем Joint Hawkes без каких-либо фиксаций — это даёт опорную точку $(\hat{\lambda}_u, \hat{\alpha})$.
2. Строим сетку из 11 значений для α_{kj} — от 0 до $\max(0.10, 2 \cdot \hat{\alpha}_{kj})$, чтобы покрыть и небольшие, и удвоенные относительно опорной точки значения.
3. На каждом значении сетки фиксируем α_{kj} и переобучаем оставшиеся параметры

модели для этого целевого канала: per-user множители λ_u и два других коэффициента строки $\alpha_{k,\cdot}$.

4. Считаем для каждой из 11 точек train NLL и test NLL — так получается profile-кривая для одной ячейки матрицы.

Чтобы оптимизатор не зависал в локальных минимумах при фиксации, идём по сетке последовательно от точки, ближайшей к опорной, и каждый следующий fit стартует с параметров предыдущего. Затем делаем два дополнительных прохода по уже найденным точкам с обновлённой опорной точкой. Это убирает дискретные скачки в profile NLL из-за неудачной инициализации.

Полная 3×3 картина для train NLL приведена на рис. 22, для test NLL — на рис. 23.

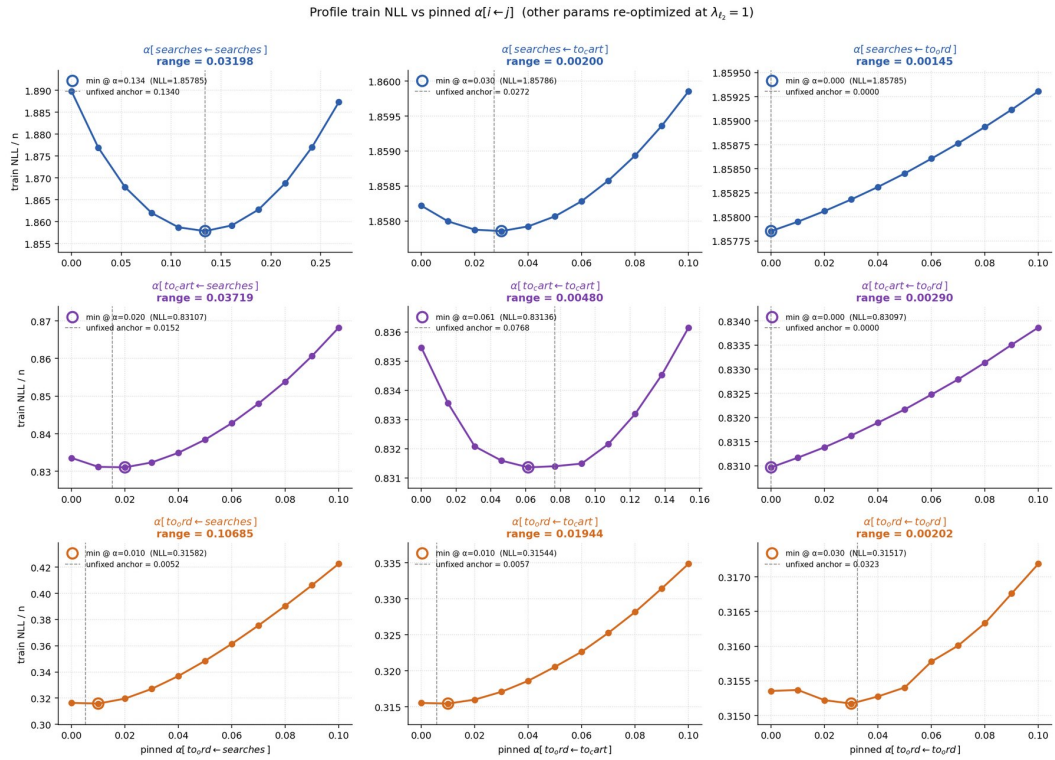


Рис. 22: Train NLL profile по 9 коэффициентам

Топология профилей характеризуется тремя группами:

- Жёсткие коэффициенты ($\text{rangeNLL} \geq 0.03$): $\text{to_ord} \leftarrow \text{searches}$ (range test NLL 0.103), $\text{searches} \leftarrow \text{searches}$ (0.096), $\text{to_art} \leftarrow \text{searches}$ (0.031). Здесь данные реально различают значения коэффициента — profile NLL имеет ярко выраженный минимум.
- Плоские коэффициенты ($\text{rangeNLL} \leq 0.005$): вся колонка $\alpha[* \leftarrow \text{to_ord}]$ плюс $\alpha[\text{searches} \leftarrow \text{to_art}]$ и $\alpha[\text{to_ord} \leftarrow \text{to_ord}]$ (range 0.002). Данные на эти значения практически не реагируют — profile NLL почти константа.

Profile test NLL vs pinned $\alpha[i \leftarrow j]$ (other params re-optimized at $\lambda_2 = 1$)

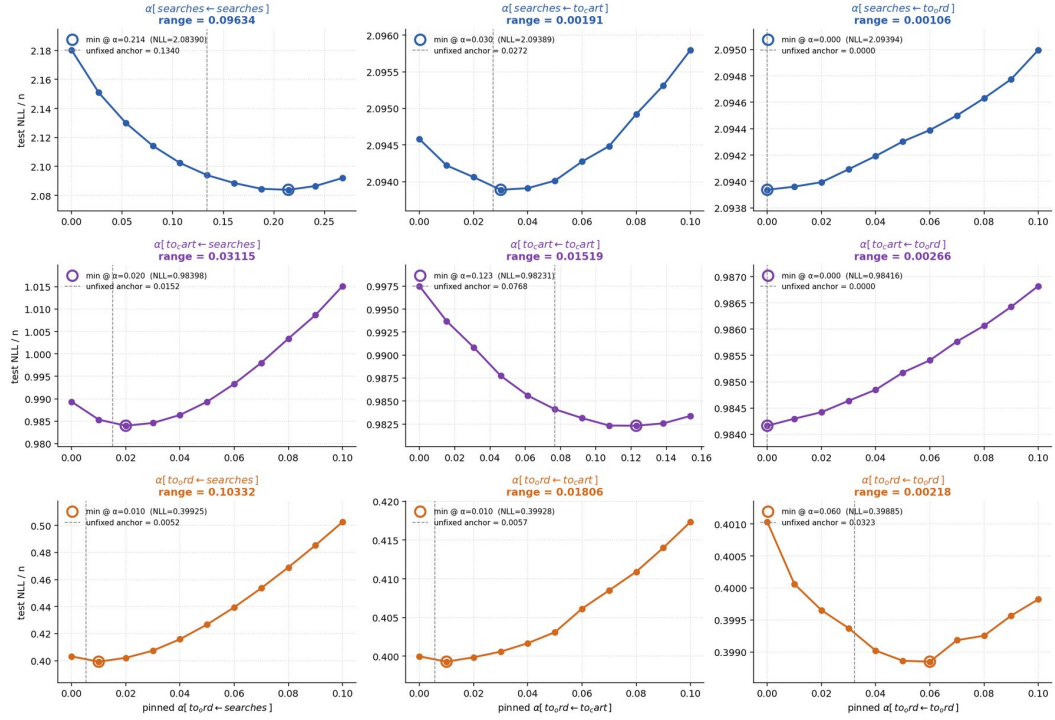


Рис. 23: Test NLL profile по 9 коэффициентам

- Промежуточные: $\text{to_ord} \leftarrow \text{to_cart}$ (0.018), $\text{to_cart} \leftarrow \text{to_cart}$ (0.015).

Топология не зависит от prior 'а: контрольный эксперимент без регуляризации ($\lambda_{\ell_2} = 0$, $\lambda_{\alpha} = 0$) даёт практически те же range 'и (с расхождением во втором-третьем знаке). То есть профили — это свойство геометрии данных, а не выбор регуляризации.

Особенно интересно, что диагональный $\alpha[\text{to_ord} \leftarrow \text{to_ord}]$ оказался плоским, несмотря на то, что в линейном фите получает ненулевое значение ~ 0.03 . Причина простая: собственная история диагонального канала $z_{u,t}^{(k)}$ в среднем пропорциональна $\lambda_u^{(k)} \cdot b_t$ — то есть тому же сигналу, что и бейзлайн-часть $\lambda_u \cdot b_t$. Два почти коллинеарных регрессора сложно разделить: оптимизатор может объяснить интенсивность как за счёт большого λ_u и маленького α_{kk} , так и наоборот — и обе комбинации дадут практически одинаковую $\hat{\lambda}$. Для внедиагональных коэффициентов история относится к другому каналу, не коллинеарному бейзлайну, поэтому такой проблемы нет, и значение восстанавливается устойчиво. То, что неидентифицируемость возникает именно на диагонали разреженного канала, — известный теоретический результат [4, 5].

В литературе эта же проблема обсуждается в двух подходах:

- Васру, Bompaire, Gaïffas, Muzy [4] предлагают отдельную регуляризацию для диагональных (ℓ_1 -штраф) и внедиагональных (trace norm) элементов, именно потому, что диагональ поглощает массу бейзлайна.
- Achab, Васру, Gaïffas, Mastromatteo, Muzy [5] отказываются от MLE в пользу integrated cumulants, чтобы обойти неидентифицируемость диагонали и бейзлайна на коротких выборках.

В нашей работе мы не используем эти методы напрямую — мы признаём диагональ слабо идентифицируемой и формализуем это через интервалы значений вместо точечной оценки (раздел 6.5).

Эмпирическая закономерность: качество идентификации диагонального коэффициента падает по мере разрежения соответствующего целевого канала. На богатом событиями `searches` диагональ выделяется хорошо ($\text{range NLL } 0.096$), на промежуточном `to_cart` — умеренно (0.015), на разреженном `to_ord` — практически плоский профиль (0.002). Это согласуется с интуицией: чтобы оптимизатор смог разделить вклад собственного history-state и бейзлайна, нужно достаточное число событий целевого канала.

Сравнение train и test профилей подтверждает ту же картину. Для внедиагональных коэффициентов и для self- α богатого канала `searches` train и test profiles совпадают по топологии и положению минимума — данных хватает, чтобы оценка не была артефактом обучения. У слабо идентифицируемых диагональных коэффициентов (в первую очередь у $\alpha[to_{ord} \leftarrow to_{ord}]$) train profile склонен показывать более узкий и глубокий минимум, чем test — то есть на разреженной диагонали модель легко переобучается под тренировочный сигнал, а её точечная оценка определяется в большей мере регуляризатором, чем данными.

6.5 Доверительные интервалы коэффициентов и финальная матрица

Из плоского profile NLL следует, что точечная оценка $\alpha[i, j]$ для плоских коэффициентов малоинформативна. Корректнее говорить интервалом: значение лежит где-то в широком диапазоне, и модель не различает точки внутри него.

Доверительный интервал для каждой ячейки α_{kj} мы строим через выигрыш Хоукс-модели над Personalized GP бейзлайном этого канала. Обозначим $\text{gain} = NLL_{\text{PG}} - NLL_{\min}$, где NLL_{PG} — test NLL Personalized GP, а $NLL_{\min}$ — минимум test NLL Joint Hawkes по профилю для α_{kj} (без регуляризации, чтобы убрать эффект prior’a). Допустимыми считаем такие α_{kj} , при которых модель сохраняет хотя бы 90% этого выигрыша, и берём непрерывный диапазон вокруг минимума профиля, где это условие выполнено — его границы и есть α_{lo} , α_{hi} , представительная точка — середина α_{mid} . Чем шире интервал, тем менее информативна точечная оценка коэффициента.

Численные результаты:

Важная оговорка: значения α_{hi} со звёздочкой (*) — это правые края сетки, на которых profile NLL ещё не превысил 10%-порог. То есть для этих трёх ячеек наши эксперименты до правой границы интервала не дошли, и фактический верхний предел допустимых значений выше указанного. По принципу построения сетки (см. 6.4) мы шли до $2 \cdot \alpha_{\text{anchor}}$, и этого диапазона на диагонали оказалось мало.

Три ячейки верхнего треугольника (`searches` $\leftarrow$ `to_cart`, `searches` $\leftarrow$ `to_ord`, `to_cart` $\leftarrow$ `to_ord`) имеют плоские профили с $\text{range} \leq 0.003$ в NLL — данные на них не

10%-gain intervals over Personalized GP baseline (threshold = $\min + 0.1 \cdot (\text{PG} - \min)$, unregularized fit, $\lambda_{t_2} = 0$)

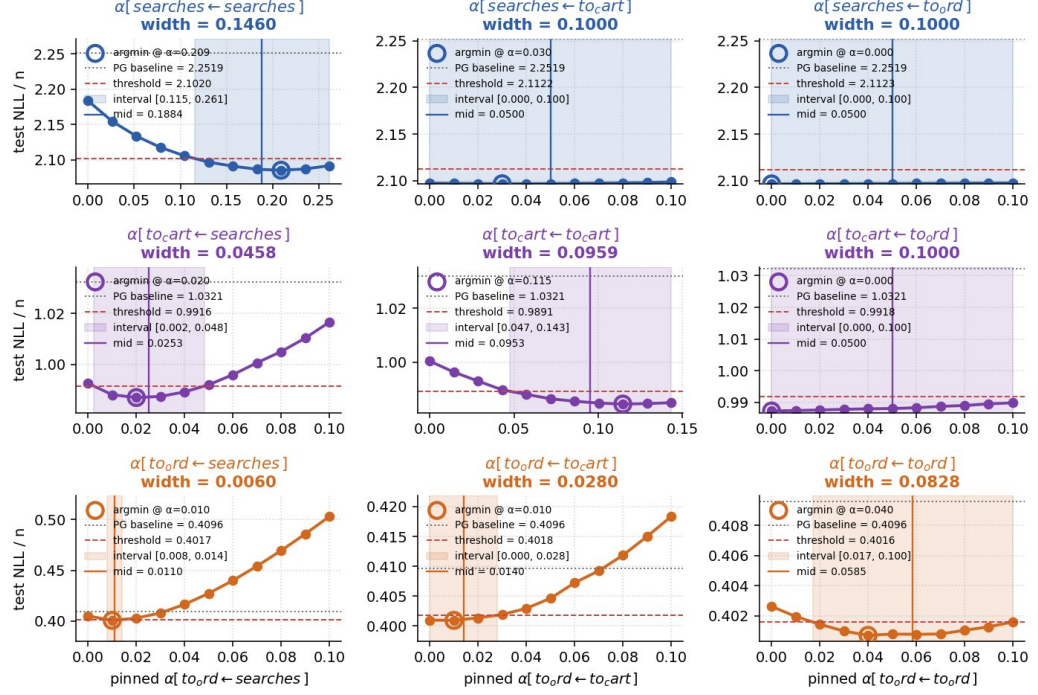


Рис. 24: Интервалы 10%-gain для 9 коэффициентов

target $\leftarrow$ source	α_{lo}	α_{hi}	α_{mid}
searches $\leftarrow$ searches	0.115	0.261*	0.188
to_cart $\leftarrow$ searches	0.002	0.048	0.025
to_cart $\leftarrow$ to_cart	0.047	0.143*	0.095
to_ord $\leftarrow$ searches	0.008	0.014	0.011
to_ord $\leftarrow$ to_cart	0.000	0.028	0.014
to_ord $\leftarrow$ to_ord	0.017	0.100*	0.059

реагируют. По convention воронки эти три коэффициента полагаем структурно нулевыми: даже на длинном 207d-train соответствующие эффекты слишком слабые, чтобы их выделить из данных, и для последующего анализа мы считаем их отсутствующими.

Финальная матрица средних точек (рис. 25):

target $\downarrow$ \ source $\rightarrow$	searches	to_cart	to_ord
searches	0.188	0	0
to_cart	0.025	0.095	0
to_ord	0.011	0.014	0.059

Матрица нижнетреугольная (по convention воронки), поэтому её собственные числа совпадают с диагональю: $|\text{eigvals}| = [0.059, 0.095, 0.188]$ для средних точек. Однако из всех предыдущих разделов мы уже знаем, что именно диагональные коэффициенты идентифицируются плохо, и для каждого из них 10%-интервал получается широким. Поэтому корректнее смотреть не точечную оценку спектрального радиуса, а его интер-

awkes interaction matrix (10%-gain midpoints, upper triangle zeroed)

$\rho(M) = 0.1884$

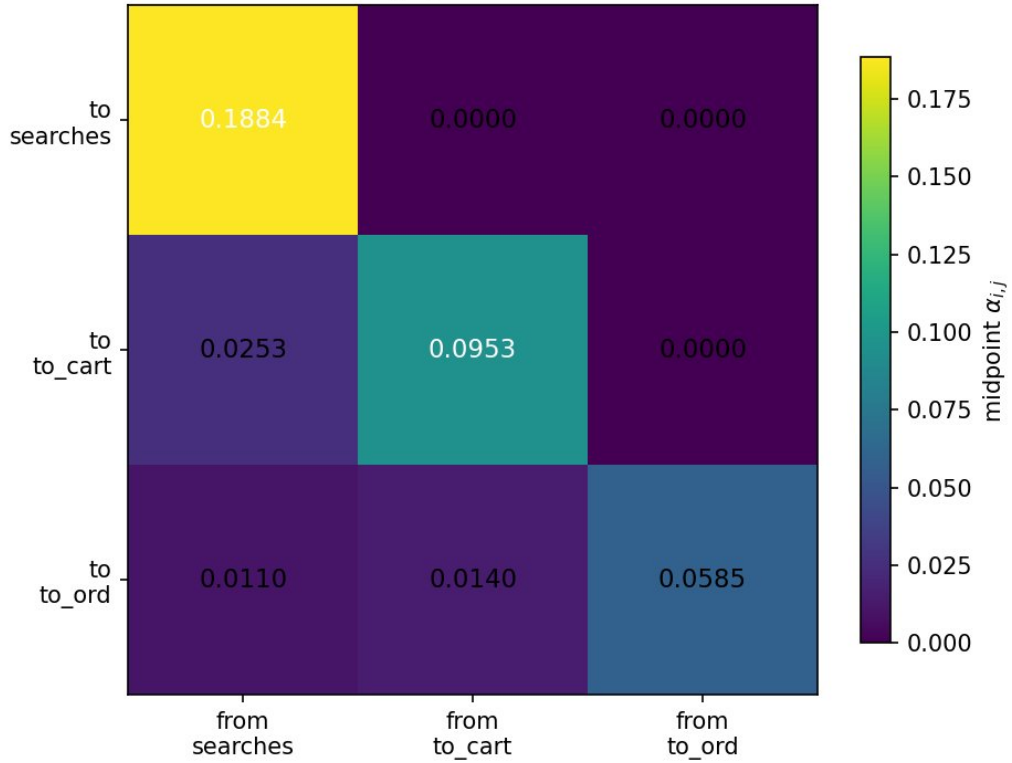


Рис. 25: Финальная Хоукс-матрица средних точек

вал, унаследованный из интервалов диагональных коэффициентов:

собственное число	α_{lo}	α_{hi}	α_{mid}
searches $\leftarrow$ searches	0.115	0.261*	0.188
to_cart $\leftarrow$ to_cart	0.047	0.143*	0.095
to_ord $\leftarrow$ to_ord	0.017	0.100*	0.059

Спектральный радиус по серединам $\rho(M) \approx 0.19$ — формально это значение можно цитировать как точечную оценку, но содержательнее интервал: левая граница около 0.12, правая упёрлась в правый край сетки и фактически выше 0.26. Даже на этой верхней границе $\rho < 1$, и Хоукс-процесс остаётся стабильным во всём измеренном диапазоне.

Финальный результат cross-channel анализа: 6-параметрическая нижнетреугольная матрица, описывающая самовозбуждающую структуру воронки **searches** $\rightarrow$ **to_cart** $\rightarrow$ **to_ord** с явно нулевыми «обратными» рёбрами. Внедиагональные коэффициенты идентифицируются устойчиво, диагональные — только в широких интервалах, что и фиксирует основное ограничение интерпретируемости модели на наших данных.

7 Глава IV. Per-user визуализация работы моделей

Для наглядности приведём пример того, как выглядит предсказанная интенсивность на четырёх конкретных пользователях — Personalized GP на рис. 26, Joint Hawkes на рис. 27.

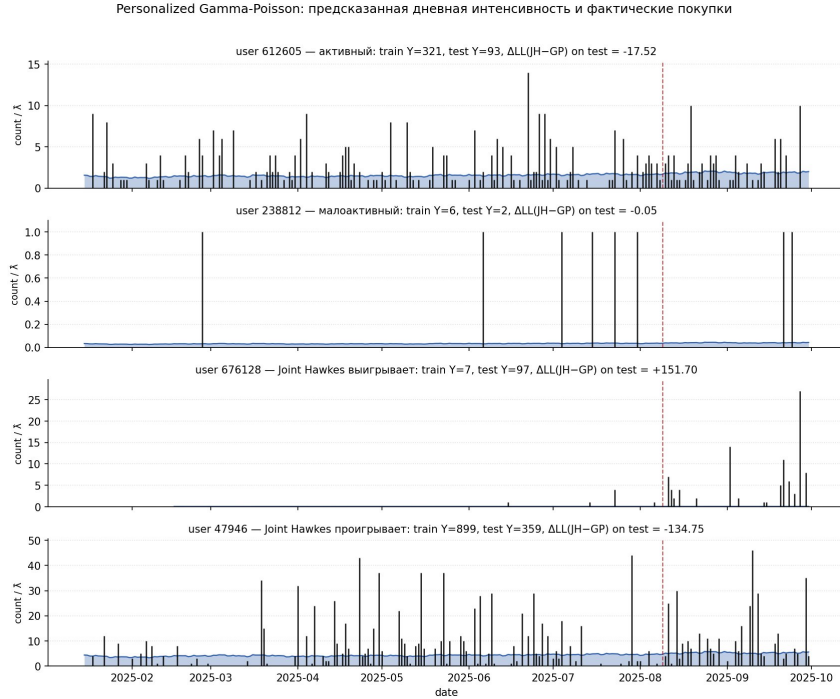


Рис. 26: Personalized GP: траектории 4 пользователей

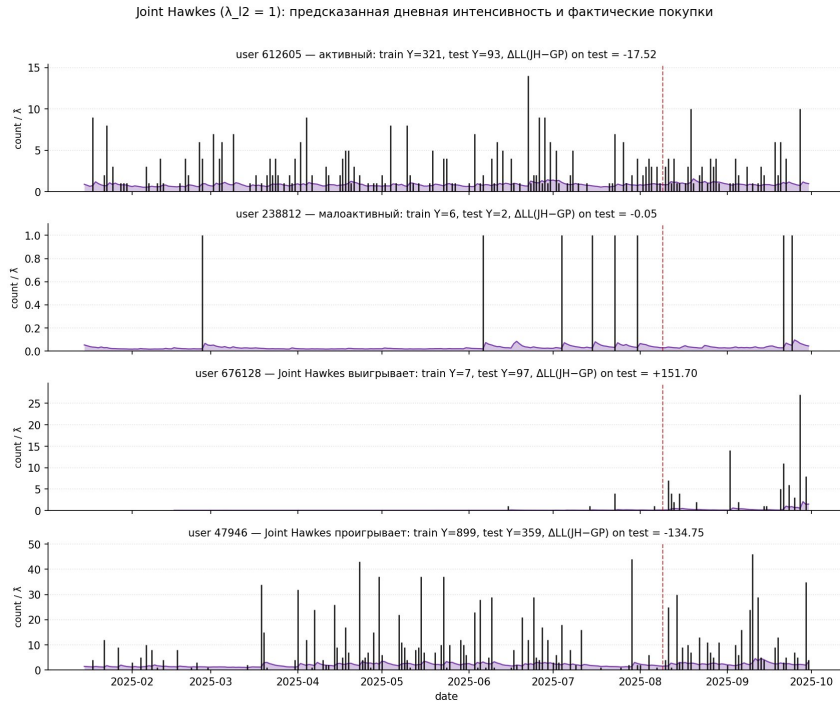


Рис. 27: Joint Hawkes: траектории 4 пользователей

8 Выводы

В работе исследованы методы моделирования пользовательской активности в сегменте e-commerce на основе пуассоновских процессов — от простых пуассоновских моделей до многомерных процессов Хоукса, с бустингом в роли контрольной модели, — на датасете из $\sim 10\text{K}$ пользователей за ~ 290 дней. Основной фокус сделан на применимости Хоукс-моделей. Основные результаты сгруппированы по основным частям работы.

Бейзлайн и Хоукс-надстройка на полном train-test (глава I). Построена последовательность из шести вероятностных моделей с переходом от Global Poisson к Joint Hawkes. Самый крупный единичный шаг ожидаемо приходится на персонализацию (Gamma-Poisson через Empirical Bayes, -0.048 в NLL): без неё модель неизбежно усредняет активных и неактивных пользователей. Содержательно более интересен следующий шаг — Хоукс-надстройка устойчиво улучшает уже сильный персонализированный бейзлайн на -0.014 в NLL. Поскольку данные табличные и наблюдений много, верхнюю планку качества ожидаемо задаёт бустинг (141 признак), но его отрыв от Хоукс-моделей небольшой: Scaled-baseline и Joint Hawkes покрывают $\sim 64\%$ и $\sim 67\%$ всего извлекаемого бустингом сигнала над Pers GP при существенно более простой и интерпретируемой структуре. На уровне отдельных каналов воронки Хоукс-надстройка тем больше покрывает зазор Pers GP $\rightarrow$ GBDT, чем плотнее канал (от $\sim 54\%$ на разреженном `to_ord` до $\sim 77\%$ на `searches`). Сравнение per-user Хоукс-моделей со Scaled-baseline и Joint Hawkes показывает, что при близком NLL на всей когорте они приходят к существенно разным внутренним декомпозициям (m_u, α) — первый намёк на трейдофф, который мы раскрываем дальше.

Поведение на коротких train-окнах (глава II). Хоукс-сигнал в данных действительно присутствует, но он слабый, и форма регуляризации существенно меняет картину. Scaled-baseline на коротких train ($n \leq 50$ дней) вырождается фактически до Personalized GP: из-за коллинеарности ЕВ-сжатой базы и редких событий целевого канала Хоукс-веса прижимаются к нулю. Joint Hawkes с явной L2-регуляризацией $(\lambda_u - 1)^2$ не вырождается, но дополнительный prior не даётся «бесплатно»: на $n \leq 21$ Joint выпирает у всех вероятностных моделей, а на промежуточных $n \in [27, 60]$ тот же $\lambda_{\ell_2} = 1$ уже излишне ограничивает модель, и Joint Hawkes на этих длинах оказывается хуже даже Personalized GP. На $n \geq 100$ обе параметризации сравниваются по test NLL, но всё ещё приходят к разным (λ_u, α) . Устойчивое преимущество Хоукс-надстройки над Personalized GP начинается с $n \approx 80$ дней. Главный практический вывод: длина train-окна имеет принципиальное значение, и универсально лучшей Хоукс-параметризации нет — выбор формы и силы регуляризации должен подстраиваться под объём доступных данных.

Структура cross-channel Хоукс-матрицы (глава III). Cross-channel матрица $\alpha \in \mathbb{R}^{3 \times 3}$ восстанавливается как осмысленная структура воронки `searches` $\rightarrow$ `to_cart` $\rightarrow$ `to_ord`. Три «обратных» коэффициента верхнего треугольника имеют плоские профили в NLL и нашими экспериментами от нуля неотличимы — мы полагаем их

структурно нулевыми по структуре воронки. Внедиагональные связи воронки положительны и идентифицируются устойчиво в смысле формы профиля, хотя их точечные значения заметно меняются с длиной train-окна (Scaled-baseline на 100d систематически недообучает всю матрицу, включая внедиагональные; Joint Hawkes с активной L2 удерживает внедиагональные даже на коротких окнах за счёт явного prior’a). Диагональные коэффициенты, особенно $\alpha[to_{ord} \leftarrow to_{ord}]$, идентифицируются плохо: profile NLL почти плоский, и точечная оценка определяется в большей мере регуляризатором, чем данными. Качество идентификации диагонали падает по мере разрежения соответствующего целевого канала — на богатом **searches** диагональ выделяется хорошо, на разреженном **to_ord** — почти не идентифицируется, что согласуется с известными результатами по многомерным процессам Хоукса [4, 5]. Перебор по $\lambda_{\ell_2} \in [0, 5]$ количественно подтверждает «переток массы» между бейзлайн-частью $\lambda_u \cdot b_t$ и Хоукс-надстройкой $\alpha^\top z$ при почти не меняющемся test NLL (размах ≤ 0.01). Это означает сверх-параметризованность модели на уровне внутренних параметров при устойчивости предсказания. После формализации идентифицируемости через 10%-gain интервалы получена финальная нижнетреугольная матрица; для её спектрального радиуса корректнее цитировать не точечную оценку, а интервал — середина ≈ 0.19 , левая граница ~ 0.12 , правая упёрлась в правый край сетки и фактически выше 0.26. Во всём измеренном диапазоне $\rho < 1$, Хоукс-процесс остаётся стабильным.

Практические выводы. Хоукс-модель пригодна для прогноза покупок в e-commerce, но с рядом оговорок. Во-первых, выбор параметризации зависит от длины train: на коротких окнах нужна явная регуляризация (Joint Hawkes с активным L2-prior’ом), на длинных — подходит любая из двух параметризаций. Во-вторых, точечные значения матрицы α нельзя цитировать как один «правильный» ответ: диагональные коэффициенты слабо идентифицируемы, и осмысленнее работать с интервалами. В-третьих, внедиагональные связи воронки интерпретируются устойчиво и могут использоваться в прикладных задачах (например, при декомпозиции эффекта на каналы в А/В-эксперименте), но и они требуют достаточной длины train. Бустинг-модели по абсолютному скору остаются лучшими, но со слабее интерпретируемой структурой — выбор между Hawkes и бустингом определяется приоритетом интерпретируемости над скорингом. Per-user визуализация показывает, что Hawkes и Personalized GP при близких метриках на всей когорте дают существенно разные индивидуальные предсказания, и это надо учитывать при прикладном применении модели.

9 Заключение

В работе построена и исследована иерархия моделей дневной интенсивности покупок пользователя — от простых пуассоновских моделей до многомерных процессов Хоукса, — а также проведено их сравнение с градиентным бустингом. Показано, что Хоукс-надстройка над персонализированным пуассоновским бейзлайном даёт устойчивый, хотя и небольшой, прирост качества, восстанавливает осмысленную структуру кросс-канальной воронки `searches` $\rightarrow$ `to_cart` $\rightarrow$ `to_ord` и при этом сохраняет интерпретируемость параметров. Основное ограничение — слабость и нестабильность Хоукс-сигнала: на коротких обучающих окнах он практически не отделяется от бейзлайна, а диагональные коэффициенты матрицы влияния идентифицируются плохо и осмысленны лишь в виде интервалов. Поставленная задача в целом решена: получены работающие модели интенсивности и количественная оценка применимости процессов Хоукса в данной задаче с честно очерченными границами этой применимости.

Естественные направления дальнейшего развития работы:

- получение более длинных историй наблюдений, на которых Хоукс-коэффициенты обеих параметризаций должны стабилизироваться, а диагональ — становиться идентифицируемой;
- на протяжении работы неоднократно отмечалось, что одни модели выигрывают у других неравномерно по пользователям: где-то прирост метрик идёт в основном за счёт активных пользователей, где-то — наоборот. Отсюда идея разбить пользователей на когорты по уровню активности и обучать на каждой когорте свою модель. В настоящей работе этот вопрос не рассматривался.

10 Использование инструментов искусственного интеллекта

При подготовке настоящей работы использовались инструменты на основе больших языковых моделей. Их применение было вспомогательным и преследовало цель ускорить рутинные операции, не связанные с получением научных результатов.

Инструмент. Claude (компания Anthropic), модель семейства Claude Opus. Работа велась интерактивно: задачи формулировались на естественном языке, результат проверялся и дорабатывался автором.

Где применялся. В основном — для редактирования текста работы: правка стиля, нахождение опечаток и оформление в L^AT_EX. Дополнительно — для написания шаблонного кода визуализаций и вспомогательных bash-скриптов.

Авторский вклад и проверка. Постановка задачи, выбор и построение моделей, проведение экспериментов, анализ результатов и выводы выполнены автором самостоятельно. Сгенерированные с помощью ИИ фрагменты текста и кода проверялись на корректность, редактировались и интегрировались автором; ответственность за итоговое содержание работы несёт автор.

Список литературы

- [1] Hawkes A. G. Spectra of some self-exciting and mutually exciting point processes // Biometrika. — 1971. — Vol. 58, no. 1. — P. 83–90.
- [2] Bacry E., Mastromatteo I., Muzy J.-F. Hawkes processes in finance // Market Microstructure and Liquidity. — 2015. — Vol. 1, no. 1. — 1550005.
- [3] Du N., Dai H., Trivedi R., Upadhyay U., Gomez-Rodriguez M., Song L. Recurrent marked temporal point processes: Embedding event history to vector // Proceedings of KDD. — 2016. — P. 1555–1564.
- [4] Bacry E., Bompierre M., Gaïffas S., Muzy J.-F. Sparse and low-rank multivariate Hawkes processes // Journal of Machine Learning Research. — 2020. — Vol. 21, no. 50. — P. 1–32.
- [5] Achab M., Bacry E., Gaïffas S., Mastromatteo I., Muzy J.-F. Uncovering causality from multivariate Hawkes integrated cumulants // Journal of Machine Learning Research. — 2018. — Vol. 18, no. 192. — P. 1–28.
- [6] Fader P. S., Hardie B. G. S., Lee K. L. RFM and CLV: Using iso-value curves for customer base analysis // Journal of Marketing Research. — 2005. — Vol. 42, no. 4. — P. 415–430.

- [7] Mei H., Eisner J. The neural Hawkes process: A neurally self-modulating multivariate point process // Advances in Neural Information Processing Systems. — 2017. — Vol. 30. — P. 6754–6764.
- [8] Friedman J. H. Greedy function approximation: A gradient boosting machine // The Annals of Statistics. — 2001. — Vol. 29, no. 5. — P. 1189–1232.